EWB CLOUD LABS

STUDENT PROJECT REPORT

BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

EC2, custom route tables, security groups and a bastion host.

```
VPC A 10.10.0.0/16
< pcx >
VPC B 10.20.0.0/16
```

AWS ARCHITECT ASSOCIATE

Name: **Paul Daniel T**

Batch: _____

Region: Asia Pacific (Mumbai) ap-south-1

Completed: 30 October 2026

1. Project overview

Mission (from the EWB Cloud Labs workbook): build a working private network connection between two VPCs and explain why every route and security rule is needed. The application server in VPC A and the workload in VPC B stay private (no public IPv4). VPC peering lets them talk over their private IPv4 addresses, and one public bastion host in VPC A is used for administration.

Item	Value
Student	Paul Daniel T
Project	EWB Cloud Labs, Project 01: AWS VPC Peering (AWS Architect Associate)
Region / AZ	Asia Pacific (Mumbai) ap-south-1, Availability Zone ap-south-1a
Build and test date	30 October 2026 (all instances launched and terminated the same day)
Instances	Amazon Linux 2023, t3.micro, x86_64 (AMI al2023-ami-2023.12.20261029.0-kernel-6.18-x86_64)
Tags on every resource	Project = EWB-VPC-Peering, Student = Paul Daniel T

What this report demonstrates

- Non-overlapping VPC and subnet CIDR planning (10.10.0.0/16 and 10.20.0.0/16).
- Public and private subnets with explicit route table associations.
- An Active peering connection, forward and return routes, and inbound rules limited to the lab hosts.
- A failed test before peering, successful two-way tests after peering, and a controlled failure with recovery.
- Complete cleanup, so nothing keeps running or billing.

Architecture

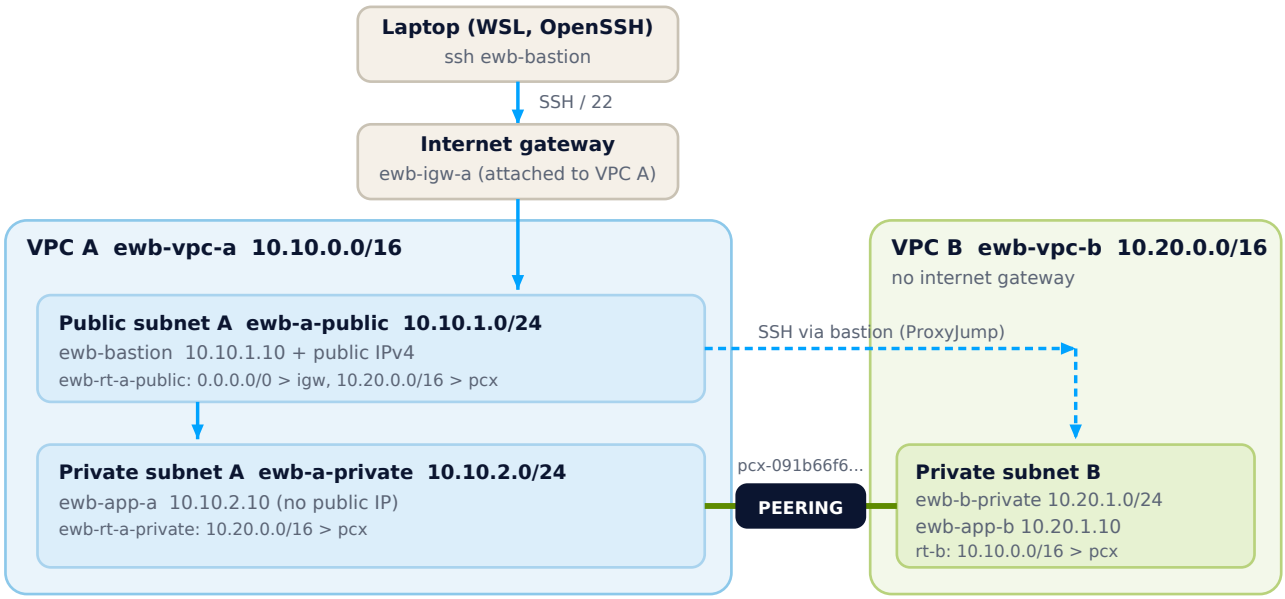


Diagram is conceptual. Administration path: laptop to bastion to App-A or App-B using SSH ProxyJump. Project traffic: App-A and App-B over private IPs through the peering connection. Peering does not give VPC B internet access through VPC A.

2. Address plan and resource inventory

Resource (Name tag)	VPC / subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A / public subnet	10.10.1.0/24
ewb-a-private	VPC A / private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B / private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4 13.201.33.173
ewb-app-a	ewb-a-private	10.10.2.10, no public IP
ewb-app-b	ewb-b-private	10.20.1.10, no public IP

The two VPC ranges do not overlap, and each /24 subnet sits inside its own VPC range. Different subnet ranges alone would not fix overlapping VPC CIDRs, and overlapping VPC CIDRs would make peering creation fail.

Actual resource IDs (recorded during the build)

Resource	Name	ID
VPC A	ewb-vpc-a	vpc-05755e24e43c88123
VPC B	ewb-vpc-b	vpc-03a912c6545b0888b
Subnet	ewb-a-public	subnet-0ec8115637feab408
Subnet	ewb-a-private	subnet-09fa25bee31c76335
Subnet	ewb-b-private	subnet-0d04409ab6609f69d
Internet gateway	ewb-igw-a	igw-0cca9215a88d719c7
Route table	ewb-rt-a-public	rtb-036f6d9d07f5482ef
Route table	ewb-rt-a-private	rtb-0a0df91264a206614
Route table	ewb-rt-b-private	rtb-049c6d698d819f4f2
Security group	ewb-sg-bastion	sg-0e75dfc9e5308bf42
Security group	ewb-sg-app-a	sg-028e2a491148c9cde
Security group	ewb-sg-app-b	sg-0b338de2b63d542c1
Key pair	ewb-lab (RSA)	key-0c28418b005fc81af
Instance	ewb-bastion	i-00dd6f21e3e7faa4e
Instance	ewb-app-a	i-07fbbac41111e3648
Instance	ewb-app-b	i-090ea2888703cec55
Peering connection	ewb-a-b-peer	pcx-091b66f68d80e321a

Privacy: the AWS account ID and the student's home public IP address are blacked out in screenshots. No private key content appears anywhere in this report.

3. Network build

The VPCs were created with the "VPC only" option so every subnet and route is controlled explicitly. Subnets all use ap-south-1a and auto-assign public IPv4 is off everywhere (the bastion gets its public IP at launch).

3.1 VPCs and subnets

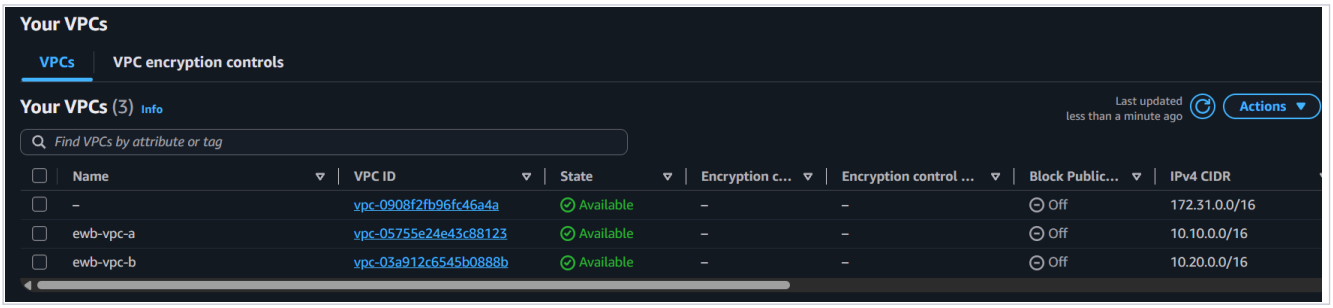


Figure 1 (S1) | **Host / console:** AWS Console, VPC, Your VPCs | **Test:** both lab VPCs created | **Result:** ewb-vpc-a 10.10.0.0/16 and ewb-vpc-b 10.20.0.0/16 are Available; the CIDRs do not overlap

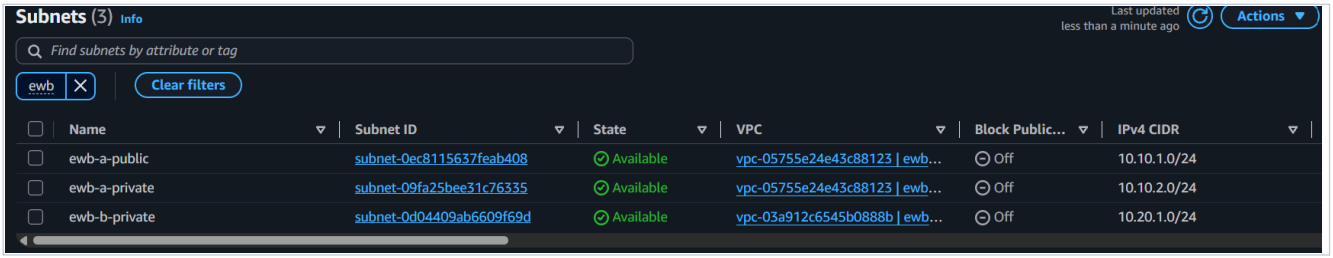


Figure 2 (S2) | **Host / console:** VPC, Subnets (filter: ewb) | **Test:** three subnets created | **Result:** ewb-a-public 10.10.1.0/24 and ewb-a-private 10.10.2.0/24 in VPC A (vpc-05755...), ewb-b-private 10.20.1.0/24 in VPC B (vpc-03a91...); all Available

3.2 Internet gateway (VPC A only)



Figure 3 (S3) | **Host / console:** VPC, Internet gateways, ewb-igw-a | **Test:** gateway attached | **Result:** State Attached to vpc-05755e24e43c88123 (ewb-vpc-a); Project and Student tags present

3.3 Route tables and explicit subnet associations (Checkpoint 01)

Public route table: explicit association with ewb-a-public and a default route to the internet gateway. Private route tables: explicit associations and only their local route, with no internet default route.

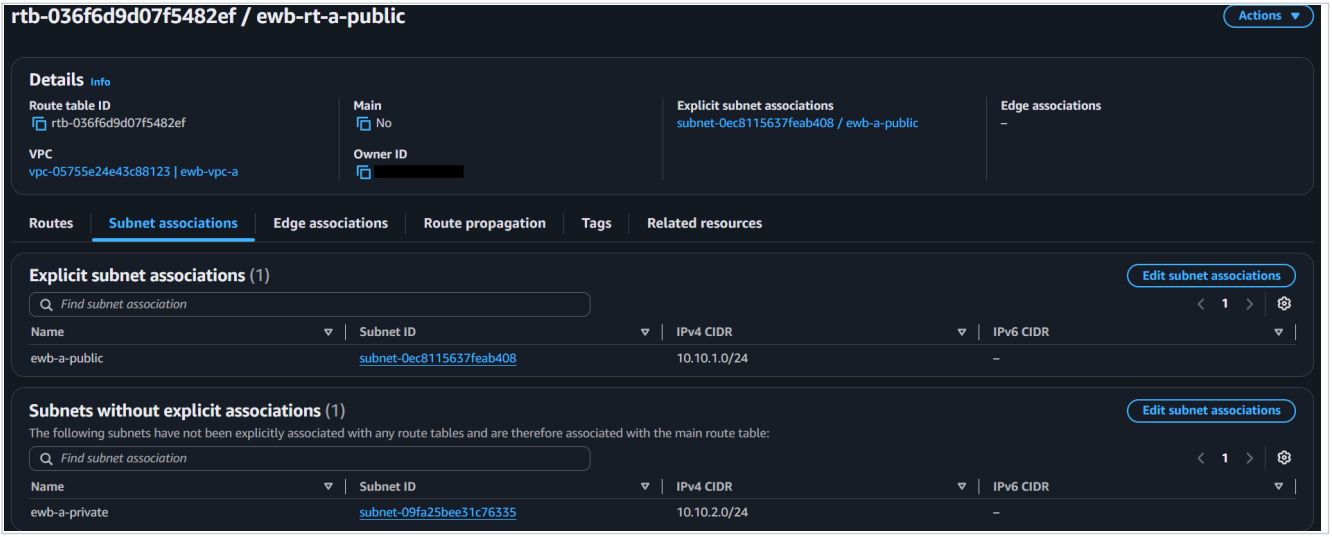


Figure 4 (S4a) | **Host / console:** ewb-rt-a-public, Subnet associations | **Test:** explicit association | **Result:** ewb-a-public (10.10.1.0/24) is explicitly associated; at this point ewb-a-private was still on the main table

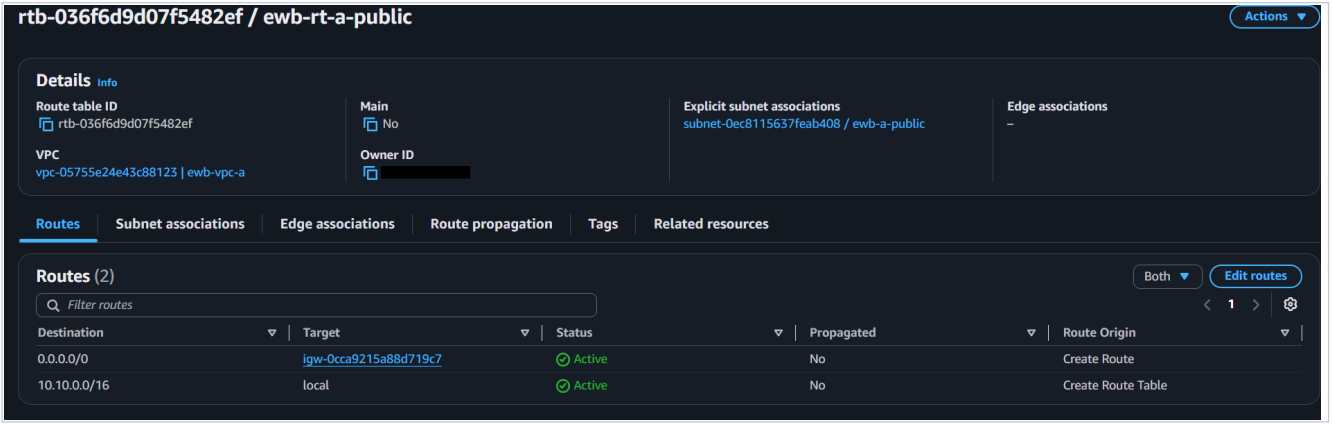


Figure 5 (S4b) | **Host / console:** ewb-rt-a-public, Routes (before peering) | **Test:** public default route | **Result:** 10.10.0.0/16 to local and 0.0.0.0/0 to igw-0cca9215a88d719c7, both Active

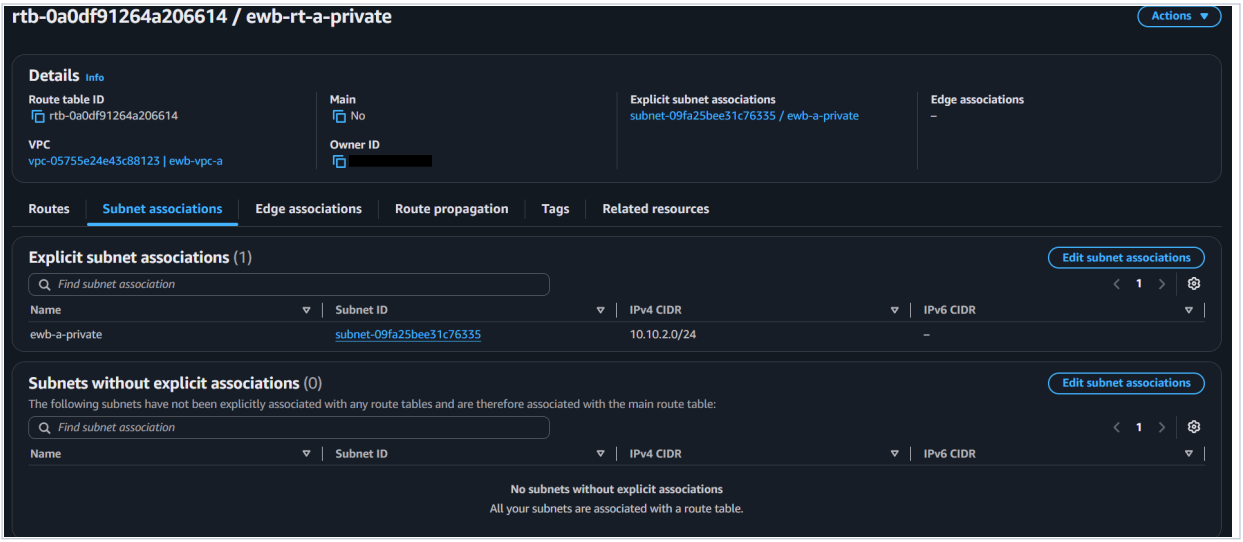


Figure 6 (S5a) | **Host / console:** ewb-rt-a-private, Subnet associations | **Test:** explicit association | **Result:** ewb-a-private (10.10.2.0/24) explicitly associated; no subnet left on the main table

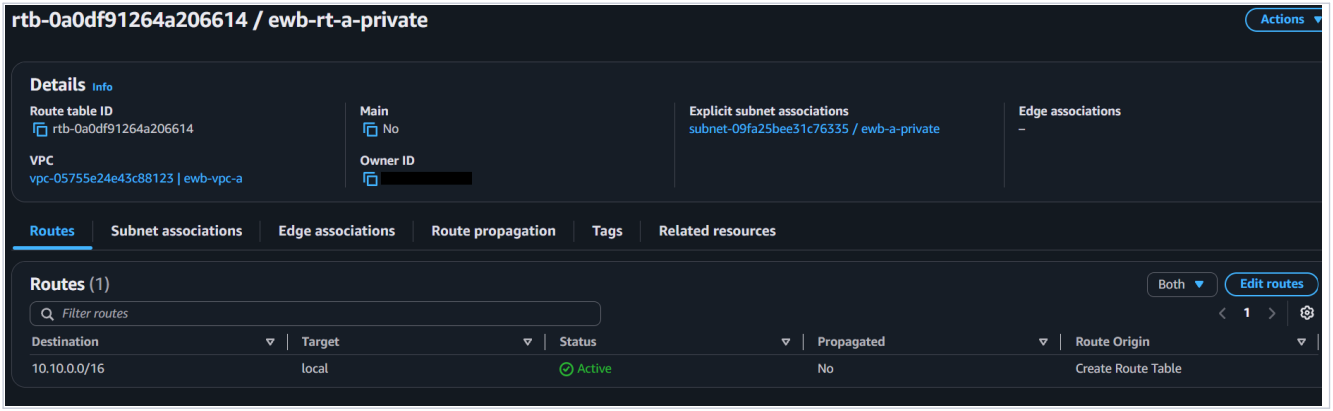


Figure 7 (S5b) | **Host / console:** ewb-rt-a-private, Routes (before peering) | **Test:** private table has no internet route | **Result:** only 10.10.0.0/16 to local

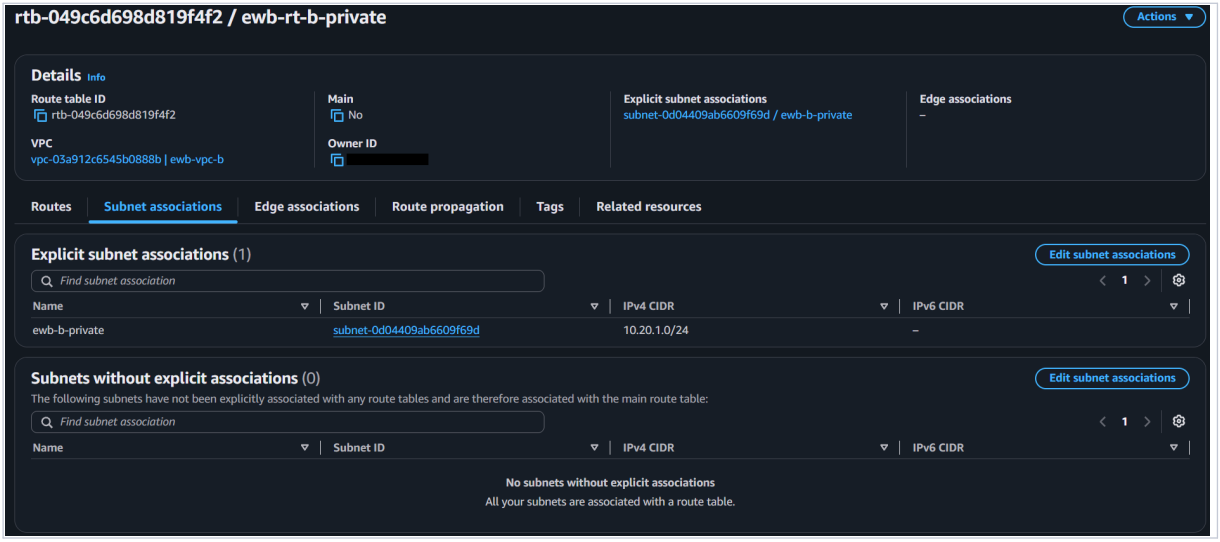


Figure 8 (S5c) | **Host / console:** ewb-rt-b-private, Subnet associations | **Test:** explicit association | **Result:** ewb-b-private (10.20.1.0/24) explicitly associated; VPC B has no internet gateway

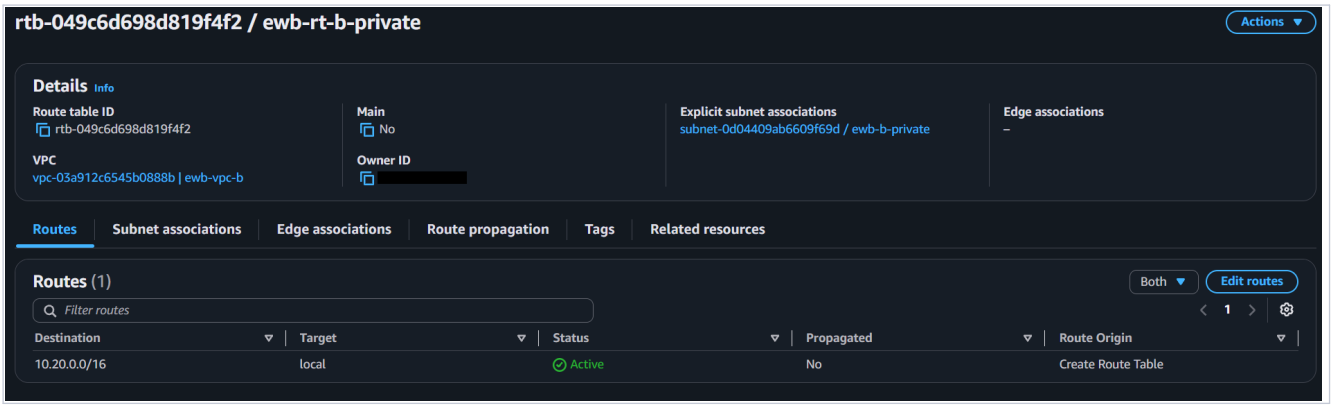


Figure 9 (S5d) | **Host / console:** ewb-rt-b-private, Routes (before peering) | **Test:** private table has local route only | **Result:** only 10.20.0.0/16 to local

4. Security groups

Inbound rules are limited to individual lab hosts (/32). Outbound rules keep the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically. Nothing is open to 0.0.0.0/0, and the default allow-all network ACLs were left in place.

Security group	VPC	Inbound rule	Why it is needed
ewb-sg-bastion	A	SSH TCP 22 from the student's public IPv4 /32	Only the student's laptop can reach the bastion
ewb-sg-app-a	A	SSH TCP 22 from 10.10.1.10/32	Admin access only from the bastion's private IP
ewb-sg-app-a	A	ICMP Echo Request from 10.20.1.10/32	Lets App-B ping App-A (reverse-path test)
ewb-sg-app-b	B	SSH TCP 22 from 10.10.1.10/32	With ProxyJump, App-B sees the bastion's private IP as the source
ewb-sg-app-b	B	ICMP Echo Request from 10.20.2.10/32	Lets App-A ping App-B (forward-path test)

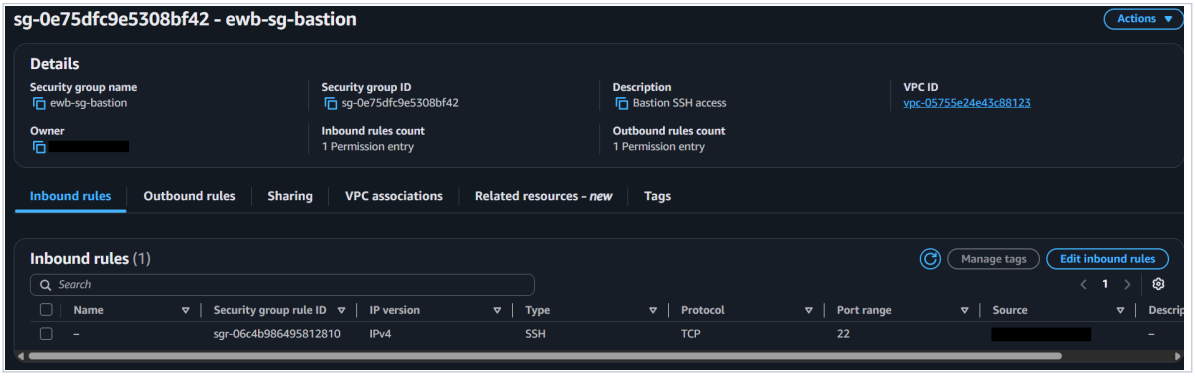


Figure 10 (S6a) | **Host / console:** ewb-sg-bastion, Inbound rules | **Test:** rule check | **Result:** SSH TCP 22 from the student's current public IP /32 (blacked out); nothing open to the world

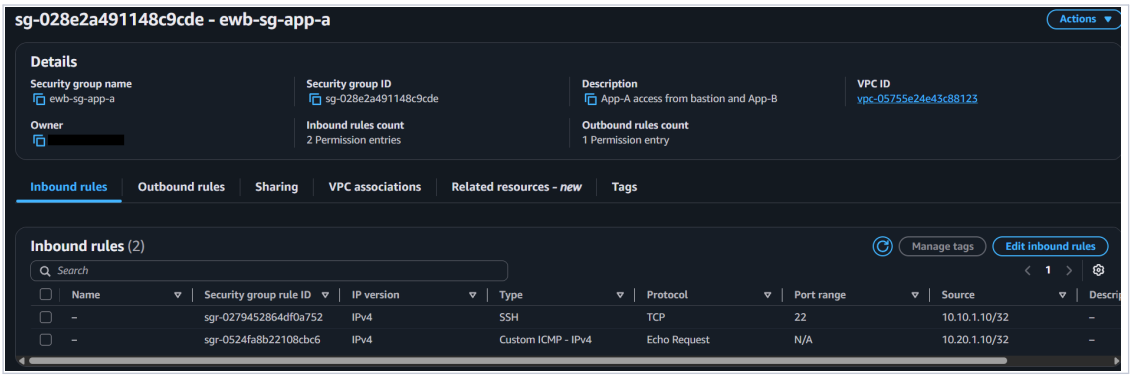


Figure 11 (S6b) | **Host / console:** ewb-sg-app-a, Inbound rules | **Test:** rule check | **Result:** SSH 22 from 10.10.1.10/32 and ICMP Echo Request from 10.20.1.10/32

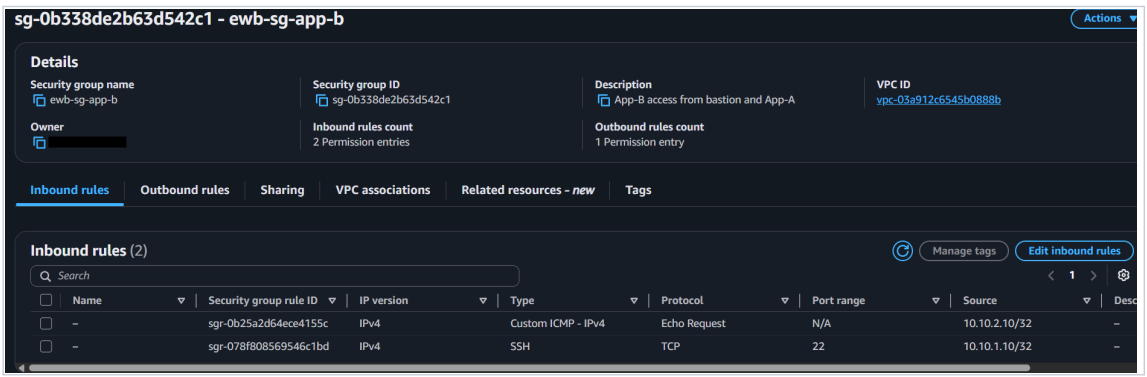


Figure 12 (S6c) | **Host / console:** ewb-sg-app-b, Inbound rules (VPC B) | **Test:** rule check | **Result:** SSH 22 from 10.10.1.10/32 and ICMP Echo Request from 10.10.2.10/32

5. Key pair and EC2 instances (Checkpoint 02)

One RSA key pair (ewb-lab) was created and its .pem file stored only on the laptop; it was never uploaded or pasted onto any instance. All three instances use Amazon Linux 2023, t3.micro, the same key pair, and a small root volume with delete-on-termination enabled. Each private IP was set during launch.

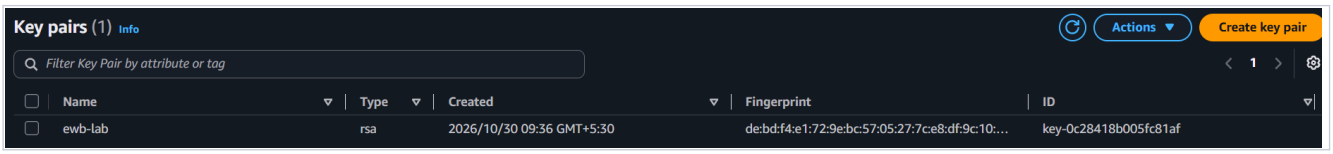


Figure 13 (S7a) | **Host / console:** EC2, Key pairs | **Test:** key pair created | **Result:** ewb-lab (RSA) listed with its fingerprint; the private key is not shown anywhere

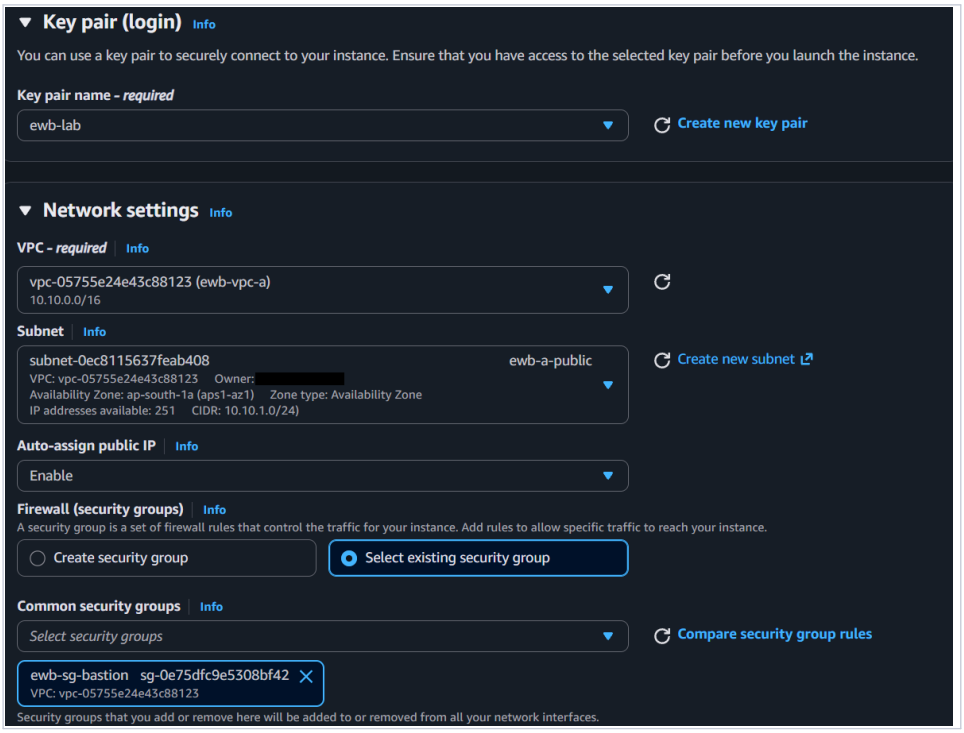


Figure 14 (S7b) | **Host / console:** EC2, Launch instance (bastion), network settings | **Test:** launch configuration | **Result:** VPC ewb-vpc-a, subnet ewb-a-public, auto-assign public IP Enable, security group ewb-sg-bastion, key pair ewb-lab

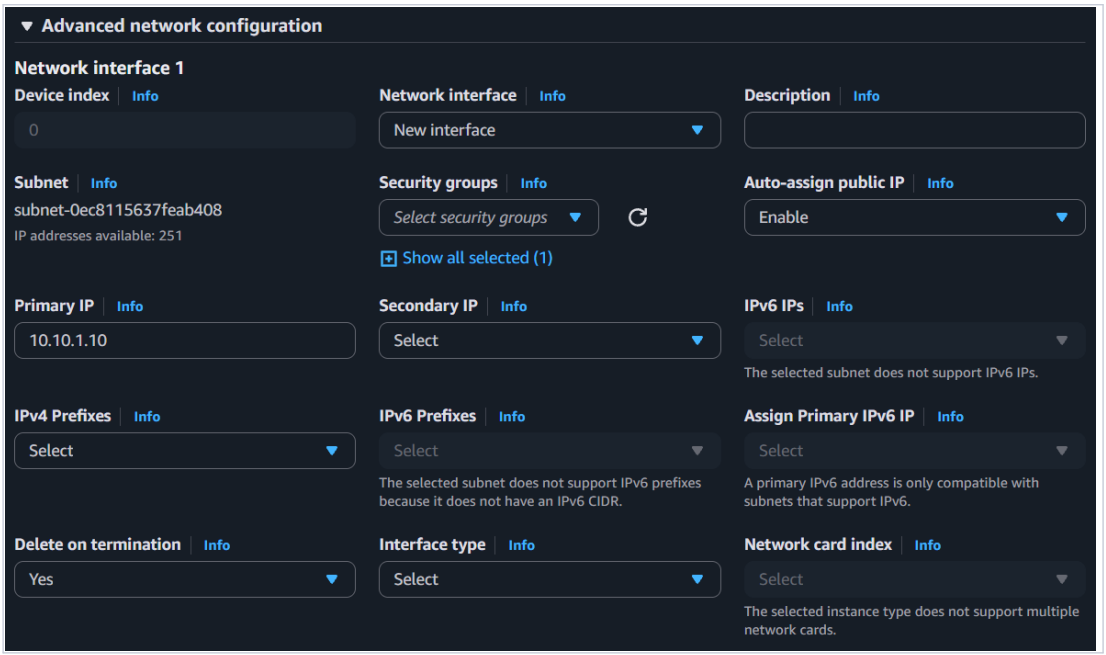


Figure 15 (S7b2) | **Host / console:** Launch instance (bastion), advanced network configuration | **Test:** fixed private IP | **Result:** primary IP 10.10.1.10, delete on termination Yes

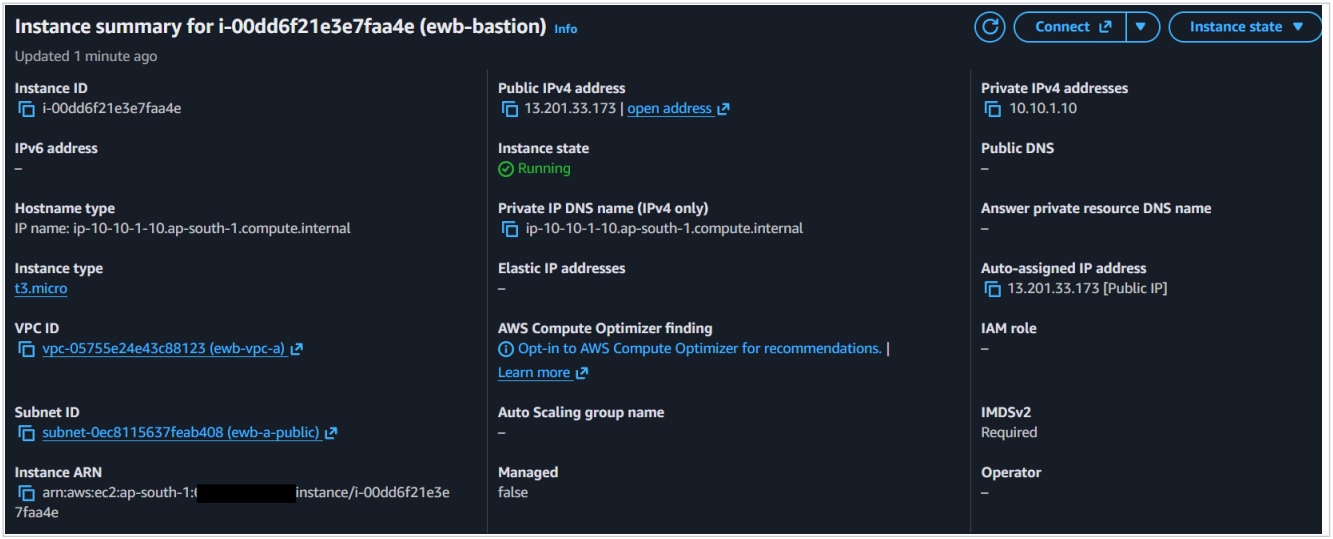


Figure 16 (S7c) | **Host / console:** ewb-bastion, Instance summary | **Test:** bastion running | **Result:** i-00dd6f21e3e7faa4e Running; public IPv4 13.201.33.173; private IPv4 10.10.1.10; ewb-vpc-a / ewb-a-public

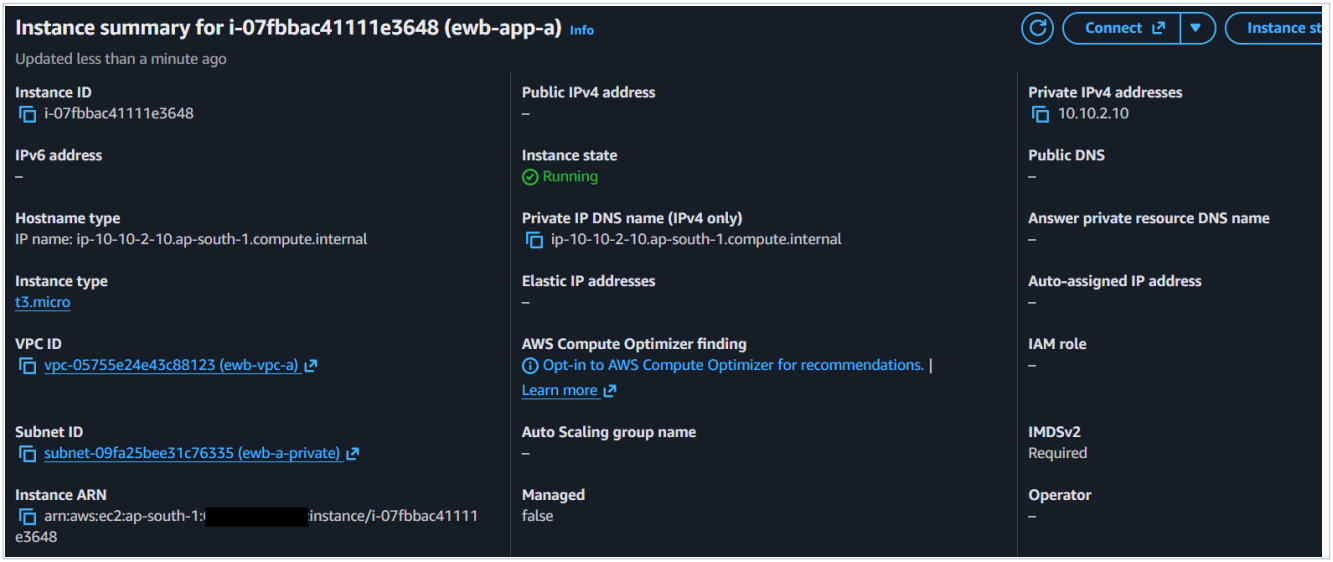


Figure 17 (S8a3) | **Host / console:** ewb-app-a, Instance summary | **Test:** App-A running and private | **Result:** i-07fbbac41111e3648 Running; no public IPv4; private IPv4 10.10.2.10; ewb-vpc-a / ewb-a-private

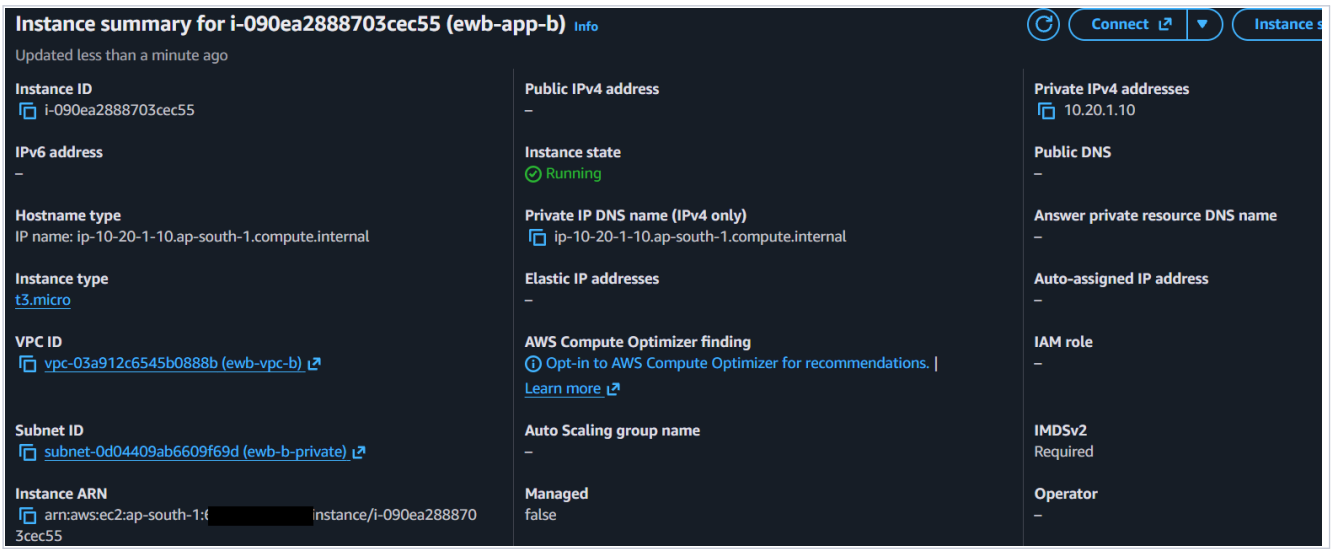


Figure 18 (S8b3) | **Host / console:** ewb-app-b, Instance summary | **Test:** App-B running and private | **Result:** i-090ea2888703cec55 Running; no public IPv4; private IPv4 10.20.1.10; ewb-vpc-b / ewb-b-private

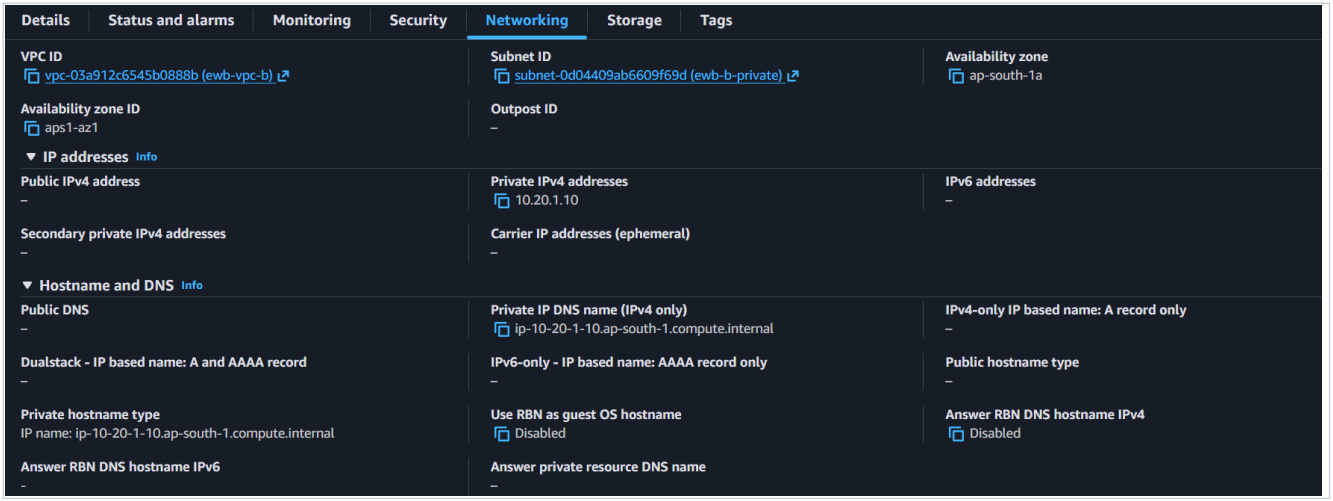


Figure 19 (S8b5) | **Host / console:** ewb-app-b, Networking tab | **Test:** network details | **Result:** no public IPv4; private IPv4 10.20.1.10; subnet ewb-b-private in ewb-vpc-b

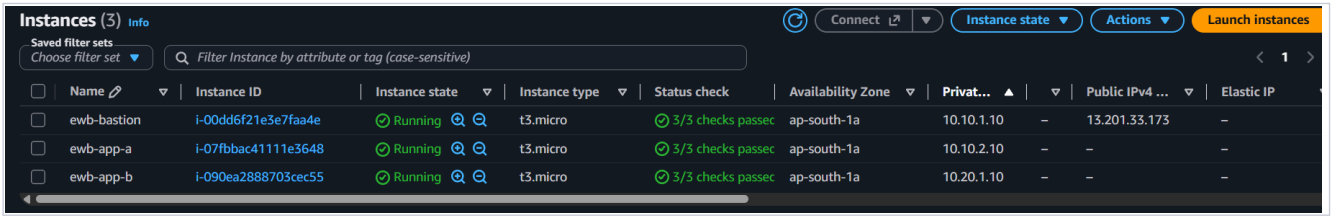


Figure 20 (S8c) | **Host / console:** EC2, Instances | **Test:** inventory of all three instances (Checkpoint 02) | **Result:** all Running with 3/3 checks passed; private IPs 10.10.1.10, 10.10.2.10, 10.20.1.10; only the bastion has a public IPv4

6. SSH access without copying keys

All SSH commands run on the laptop (WSL Ubuntu). The key file is mode 400 and the SSH config uses ProxyJump, so the bastion only forwards the connection and the key never leaves the laptop.

```
jinu@Mjolnir:~$ cp "/mnt/c/Users/Jinu G Mathew/Downloads/ewb-lab.pem" ~/.ssh/ewb-lab.pem
chmod 400 ~/.ssh/ewb-lab.pem
ls -l ~/.ssh/ewb-lab.pem
-r-------- 1 jinu jinu 1678 Oct 30 04:32 /home/jinu/.ssh/ewb-lab.pem
jinu@Mjolnir:~$
```

Figure 21 (S9a) | **Host / console:** Laptop (WSL), key file | **Test:** permission check | **Result:** ewb-lab.pem is -r-------- (chmod 400); contents are not shown

```
jinu@Mjolnir:~$ cat ~/.ssh/config
Host ewb-bastion
    HostName 13.201.33.173
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
jinu@Mjolnir:~$
```

Figure 22 (S9config) | **Host / console:** Laptop (WSL), cat ~/.ssh/config | **Test:** SSH configuration | **Result:** three hosts; ewb-app-a (10.10.2.10) and ewb-app-b (10.20.1.10) use ProxyJump ewb-bastion

```
jinu@Mjolnir:~$ ssh ewb-bastion
The authenticity of host '13.201.33.173 (13.201.33.173)' can't be established.
ED25519 key fingerprint is: SHA256:Lwz+R+NfPfFvw1z4mlcgPbX7cpqbDjWPhMpdBpE+m1k
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.201.33.173' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
```

Figure 23 (S9b) | **Host / console:** Laptop to ewb-bastion | **Test:** ssh ewb-bastion | **Result:** logged in over the internet gateway (Amazon Linux 2023 banner)

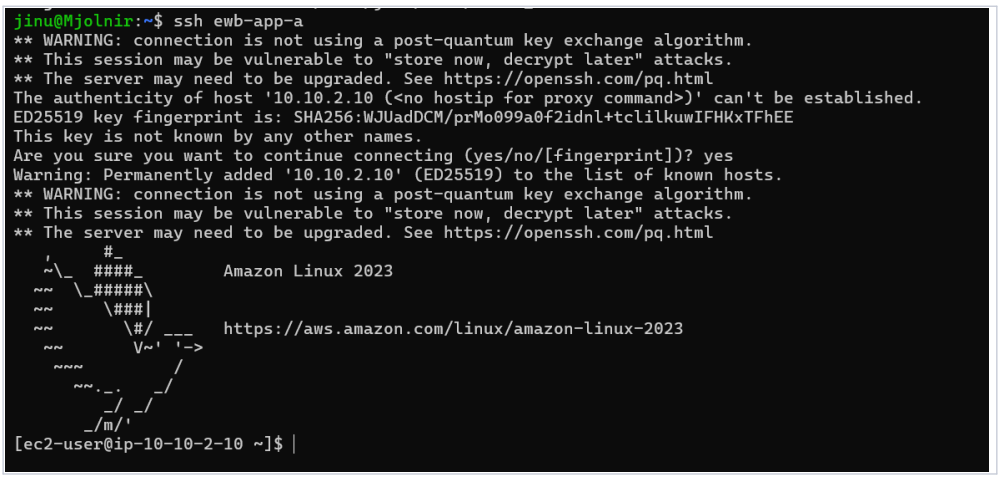


Figure 24 (S9clogin) | **Host / console:** Laptop to ewb-app-a through the bastion | **Test:** ssh ewb-app-a | **Result:** prompt ec2-user@ip-10-10-2-10 reached over VPC A's local route via ProxyJump

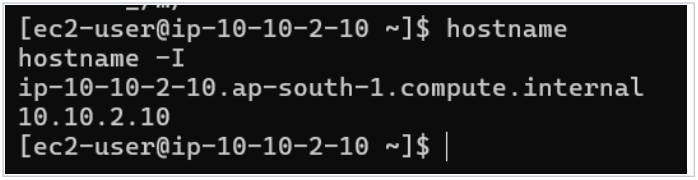


Figure 25 (S9c) | **Host / console:** ewb-app-a | **Test:** hostname and hostname -I | **Result:** ip-10-10-2-10.ap-south-1.compute.internal and 10.10.2.10

> The first connection to each host showed a "REMOTE HOST IDENTIFICATION HAS CHANGED" warning for 10.10.2.10 and later 10.20.1.10. The cause was an older saved host key for the same private IP in known_hosts from an earlier session, not a network problem. The stale entries were removed with ssh-keygen -R and the new host keys were accepted.

7. VPC peering (Checkpoint 03)

The peering connection ewb-a-b-peer was requested from VPC A (requester) to VPC B (accepter) in the same account and Region. The request must be accepted even when both VPCs belong to the same account. Peering is only the link: routes and security groups decide whether traffic can use it.

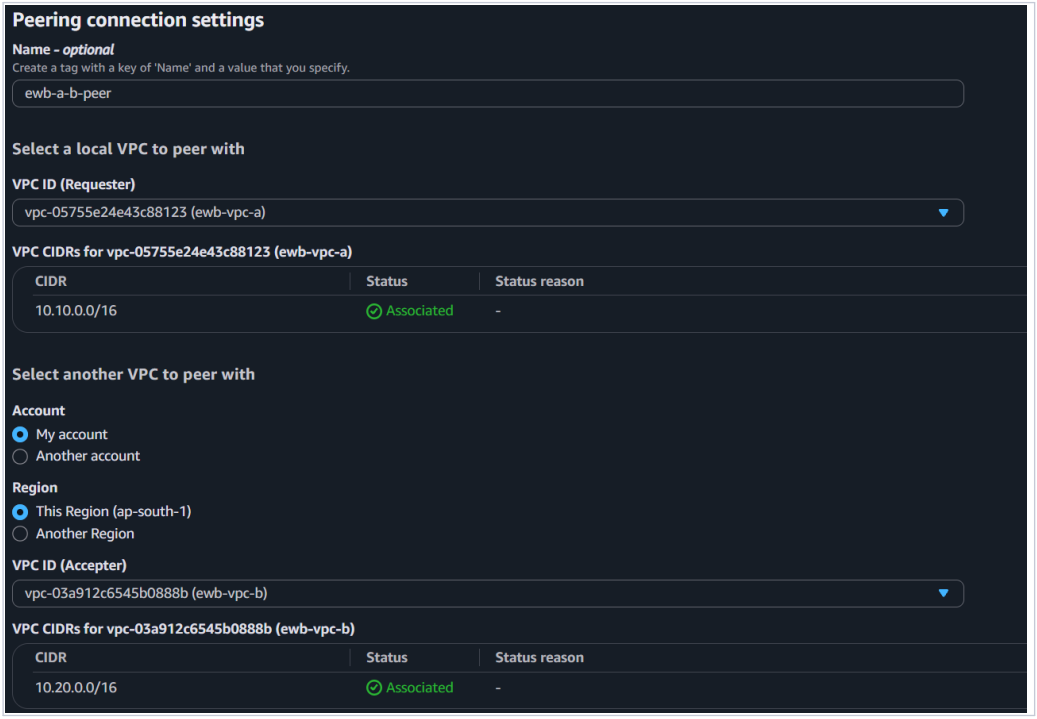


Figure 26 (S10a) | **Host / console:** VPC, Peering connections, Create | **Test:** peering request | **Result:** requester ewb-vpc-a 10.10.0.0/16, accepter ewb-vpc-b 10.20.0.0/16, My account, This Region (ap-south-1)

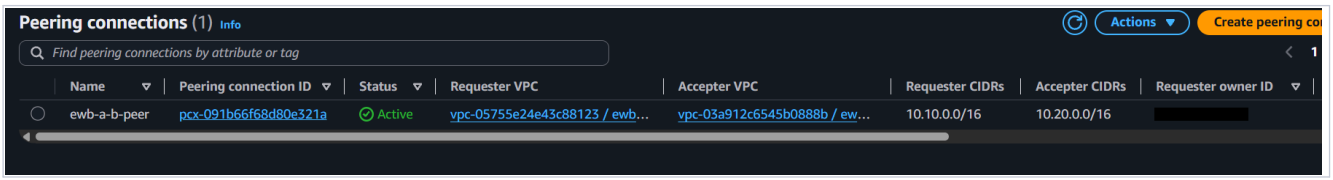


Figure 27 (S10b) | **Host / console:** VPC, Peering connections | **Test:** request accepted | **Result:** ewb-a-b-peer (pcx-091b66f68d80e321a) is Active; requester vpc-05755..., accepter vpc-03a91...; CIDRs 10.10.0.0/16 and 10.20.0.0/16

8. Routes on both sides (Checkpoint 04)

Route table (subnet)	Destination	Target	Purpose
ewb-rt-a-private (ewb-a-private)	10.10.0.0/16 / 10.20.0.0/16	local / pcx-091b...	App-A to VPC B (forward path)
ewb-rt-b-private (ewb-b-private)	10.20.0.0/16 / 10.10.0.0/16	local / pcx-091b...	Return path from App-B to VPC A
ewb-rt-a-public (ewb-a-public)	10.10.0.0/16, 0.0.0.0/0, 10.20.0.0/16	local, igw, pcx-091b...	Bastion to App-B (needed for ProxyJump)

The bastion's public subnet needs its own peering route because ProxyJump makes the bastion open a separate connection to App-B. The more specific 10.20.0.0/16 route is chosen over the internet default route. A one-way route is not enough: replies also need the route back.

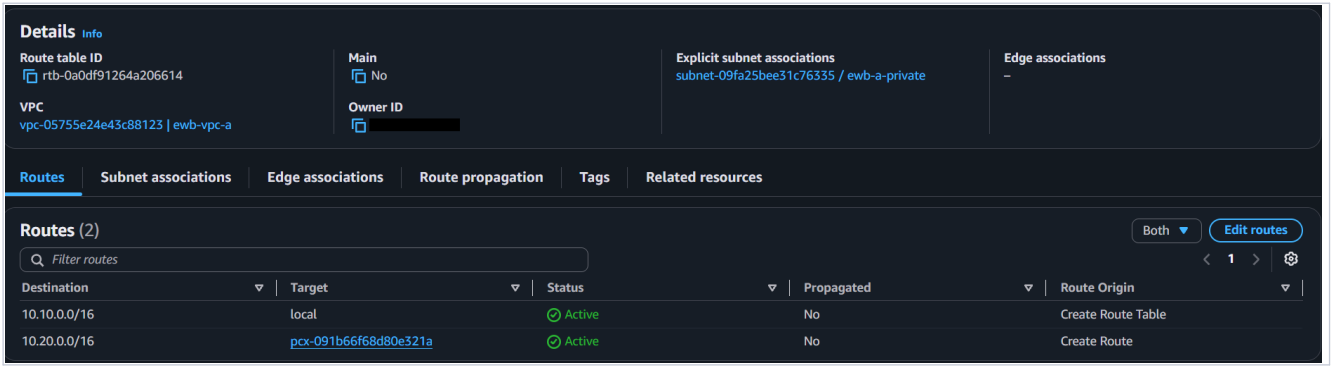


Figure 28 (S11a) | **Host / console:** ewb-rt-a-private, Routes | **Test:** forward route added | **Result:** 10.20.0.0/16 to pcx-091b66f68d80e321a, Active; local route kept

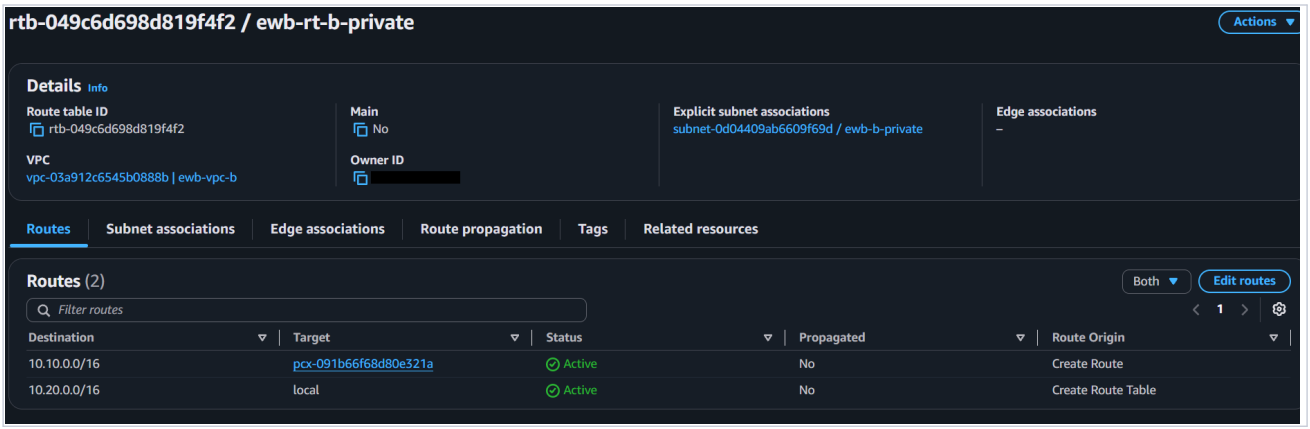


Figure 29 (S11b) | **Host / console:** ewb-rt-b-private, Routes | **Test:** return route added | **Result:** 10.10.0.0/16 to pcx-091b66f68d80e321a, Active; local route kept

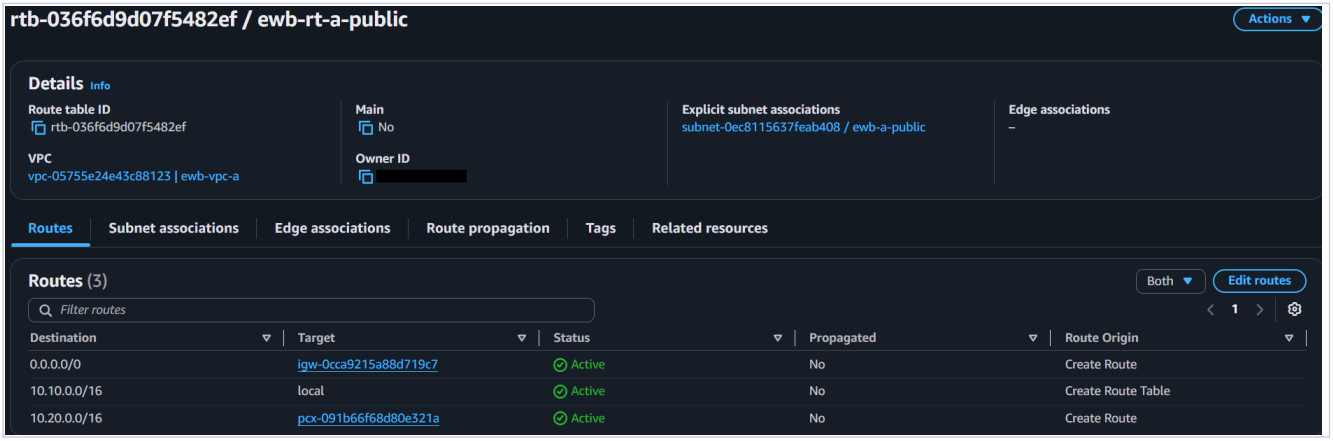


Figure 30 (S11c) | **Host / console:** ewb-rt-a-public, Routes | **Test:** bastion route added | **Result:** 0.0.0.0/0 to igw, 10.10.0.0/16 local, 10.20.0.0/16 to pcx-091b66f68d80e321a; all Active

9. Verification

Test	Where / command	Expected	Observed (figure)
01 Baseline	App-A, before peering: ping -c 4 10.20.1.10	No replies	0 of 4 received, 100% loss (S9d)
02 Forward path	App-A, after routes: ping -c 4 10.20.1.10	4 replies, 0% loss	4 of 4 received, 0% loss (S12a)
03 Reverse path	App-B, after routes: ping -c 4 10.10.2.10	4 replies, 0% loss	4 of 4 received, 0% loss (S12c)
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Prompt ip-10-20-1-10 (S12b)
05 Isolation	EC2 instance details: App-A and App-B	No public IPv4 on either host	Public IPv4 empty (S8c, S8a3, S8b3, S8b5)

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3117ms
```

Figure 31 (S9d) | **Host / console:** ewb-app-a | **Test:** Test 01: baseline ping before peering | **Result:** 4 packets transmitted, 0 received, 100% packet loss (expected failure, no peering or routes existed)

```
[ec2-user@ip-10-10-2-10 ~]$ hostname -I
ping -c 4 10.20.1.10
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.429 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.167 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.150 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.183 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3090ms
rtt min/avg/max/mdev = 0.150/0.232/0.429/0.114 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Figure 32 (S12a) | **Host / console:** ewb-app-a (10.10.2.10) | **Test:** Test 02: forward ping to App-B (10.20.1.10) | **Result:** 4 transmitted, 4 received, 0% packet loss, about 0.2 ms

```
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.350 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.171 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.155 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.239 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3083ms
rtt min/avg/max/mdev = 0.155/0.228/0.350/0.076 ms
[ec2-user@ip-10-20-1-10 ~]$
```

Figure 33 (S12c) | **Host / console:** ewb-app-b (10.20.1.10) | **Test:** Test 03: reverse ping to App-A (10.10.2.10) | **Result:** 4 transmitted, 4 received, 0% packet loss

```
[ec2-user@ip-10-20-1-10 ~]$ hostname
hostname -I
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
[ec2-user@ip-10-20-1-10 ~]$
```

Figure 34 (S12b) | **Host / console:** ewb-app-b, reached from the laptop through the bastion | **Test:** Test 04: ssh ewb-app-b, then hostname -I | **Result:** prompt ec2-user@ip-10-20-1-10; hostname -I prints 10.20.1.10

Test 05 evidence is in section 5: the Public IPv4 field is empty for App-A and App-B (S8c, S8a3, S8b3, S8b5), and tests 01 to 04 all use private destination IPs.

10. Controlled failure and recovery

Goal: show that the security group, not the network path, controls whether the ping works. Only the ICMP Echo Request rule on ewb-sg-app-b (source 10.10.2.10/32) was removed. The SSH rule, the peering connection and all routes stayed unchanged.

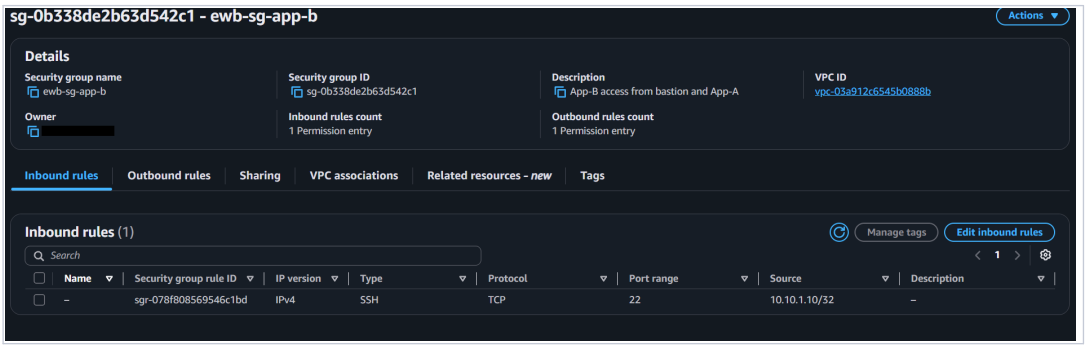


Figure 35 (S13a) | **Host / console:** ewb-sg-app-b, Inbound rules | **Test:** ICMP rule removed | **Result:** only SSH TCP 22 from 10.10.1.10/32 remains (1 rule)

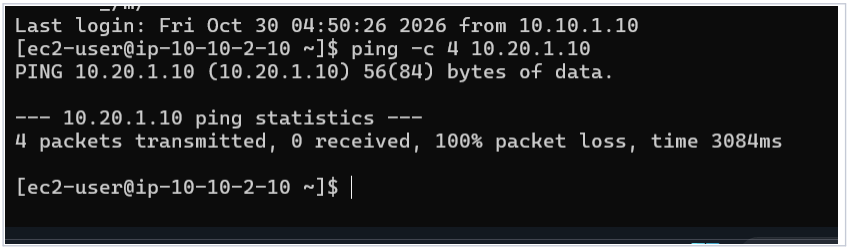


Figure 36 (S13b) | **Host / console:** ewb-app-a | **Test:** ping -c 4 10.20.1.10 with the rule removed | **Result:** 4 transmitted, 0 received, 100% packet loss, even though peering was Active and routes existed

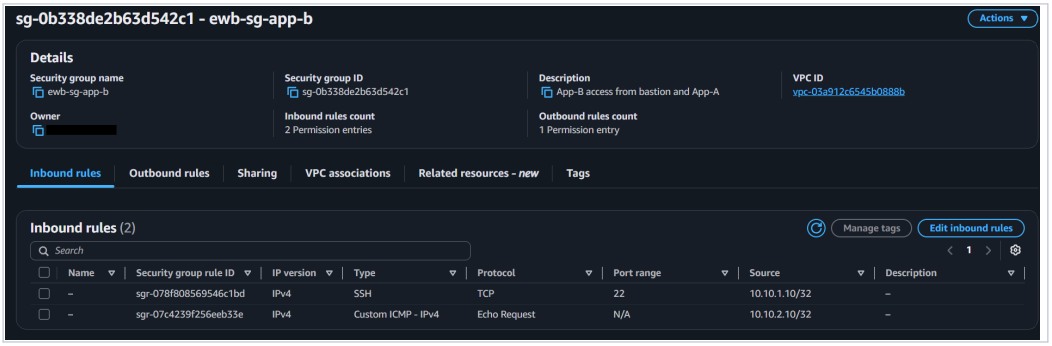


Figure 37 (S13c) | **Host / console:** ewb-sg-app-b, Inbound rules | **Test:** exact rule restored | **Result:** SSH from 10.10.1.10/32 and Custom ICMP IPv4 Echo Request from 10.10.2.10/32 (the restored rule has a new rule ID)

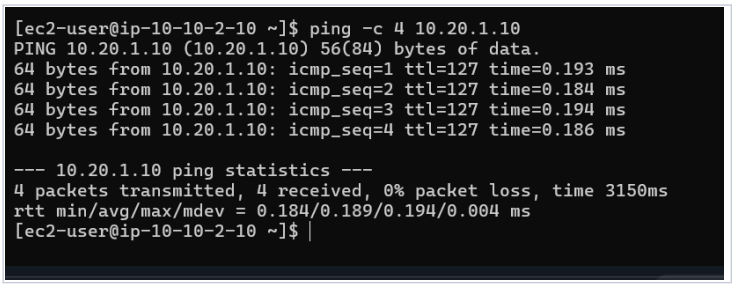


Figure 38 (S13d) | **Host / console:** ewb-app-a | **Test:** ping -c 4 10.20.1.10 after restoring the rule | **Result:** 4 transmitted, 4 received, 0% packet loss

Explanation: with the Echo Request rule gone, App-B's security group drops the incoming ping, so no reply is ever sent, even though the peering connection and both routes are fine. Restoring the identical rule brought replies straight back. This shows that Active peering alone does not guarantee a successful ping: routes and security rules must also allow the traffic.

11. Forward path, return path and viva answers

Forward and return path of the ping test

Step	Forward: App-A (10.10.2.10) to App-B (10.20.1.10)	Return: App-B to App-A
1	ewb-rt-a-private matches 10.20.0.0/16 and sends the packet to pcx-091b66f68d80e321a (local covers only 10.10.0.0/16).	ewb-rt-b-private matches 10.10.0.0/16 and sends the reply back into the peering connection.
2	The packet crosses the peering connection into VPC B.	The reply crosses the peering connection into VPC A and reaches App-A.
3	ewb-sg-app-b allows ICMP Echo Request from 10.10.2.10/32, so App-B receives it.	The security group is stateful, so the reply to a permitted request is allowed without another rule.

SSH path (laptop to App-B): the laptop reaches the bastion through the internet gateway; the bastion then opens a *separate* SSH connection to App-B. That connection leaves the public subnet using the 10.20.0.0/16 peering route, is allowed by ewb-sg-app-b (SSH from 10.10.1.10/32), and the reply returns through VPC B's 10.10.0.0/16 route.

Be ready to explain

1. Why is the return route needed? Peering does not add routes by itself. Each side's route table must know how to reach the other VPC's CIDR. Without 10.10.0.0/16 to the peering connection in VPC B, App-B has no path to send replies to App-A, so a one-way route is not enough.
2. Why is the App-B SSH source the bastion private IP? With ProxyJump the bastion opens its own connection to App-B, so App-B sees the bastion's private address (10.10.1.10) as the source, not the laptop's public IP. The rule is therefore SSH from 10.10.1.10/32.
3. Why does Active peering not guarantee a successful ping? Active only means the link exists. Traffic also needs routes in both directions and security group (and network ACL) rules that allow it. The controlled failure above proves it: peering and routes were fine, yet the ping failed when the ICMP rule was removed.
4. Can VPC B use VPC A's internet gateway over peering? No. Peering is one-to-one and non-transitive and does not forward traffic to another VPC's internet gateway. VPC B has no internet gateway and no default route, and its instance has no public IP.

12. Cleanup evidence

Everything created for this project was removed in dependency order: instances, then peering routes and the peering connection, then security groups, subnets, route tables, the internet gateway, and finally both VPCs. The key pair was deleted from AWS, and the .pem file, SSH config and saved host keys were deleted from the laptop (a local step, so there is no screenshot).

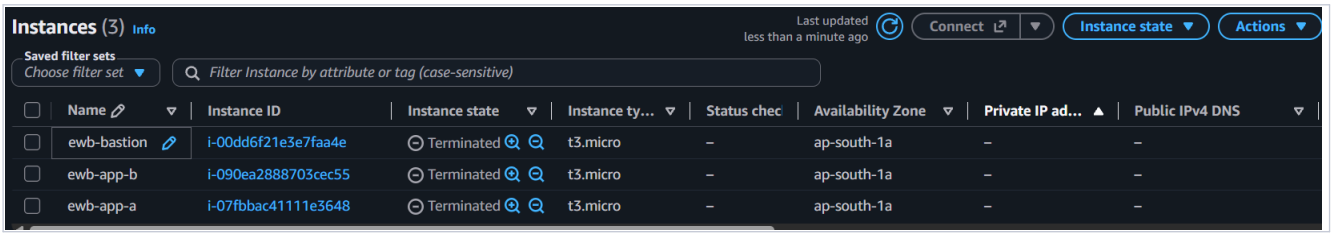


Figure 39 (C14a) | **Host / console:** EC2, Instances | **Test:** terminate the three instances | **Result:** ewb-bastion, ewb-app-a and ewb-app-b all Terminated

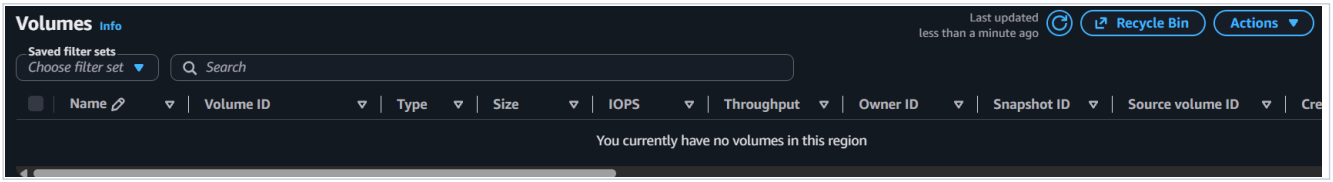


Figure 40 (C14b) | **Host / console:** EC2, Volumes | **Test:** check leftover disks | **Result:** no volumes in the Region (delete-on-termination worked)

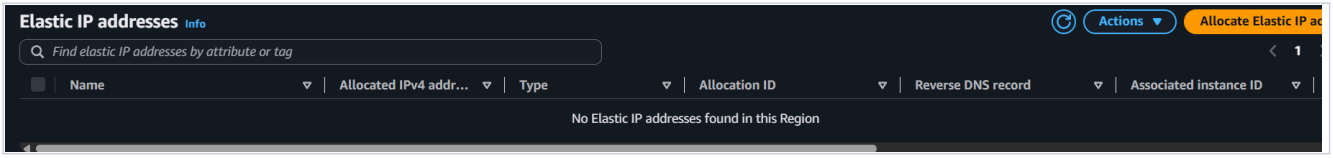


Figure 41 (C14c) | **Host / console:** EC2, Elastic IP addresses | **Test:** check for Elastic IPs | **Result:** none allocated

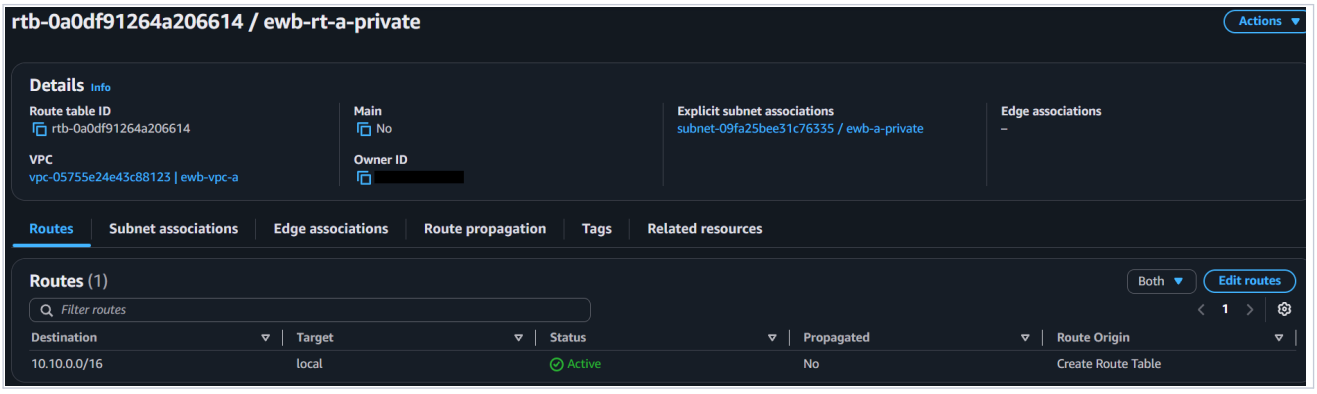


Figure 42 (C14d) | **Host / console:** ewb-rt-a-private, Routes | **Test:** peering route removed | **Result:** only 10.10.0.0/16 to local remains

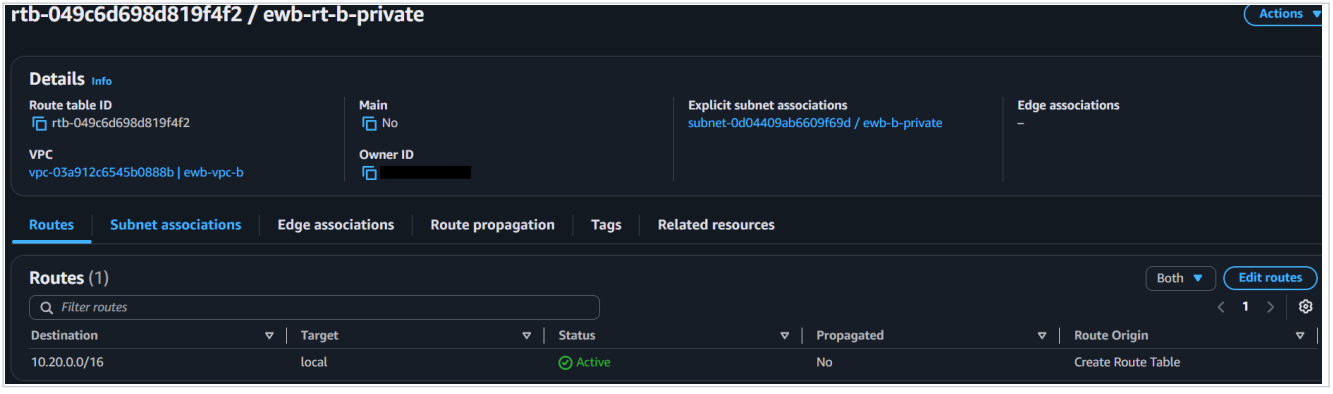


Figure 43 (C14d2) | **Host / console:** ewb-rt-b-private, Routes | **Test:** peering route removed | **Result:** only 10.20.0.0/16 to local remains

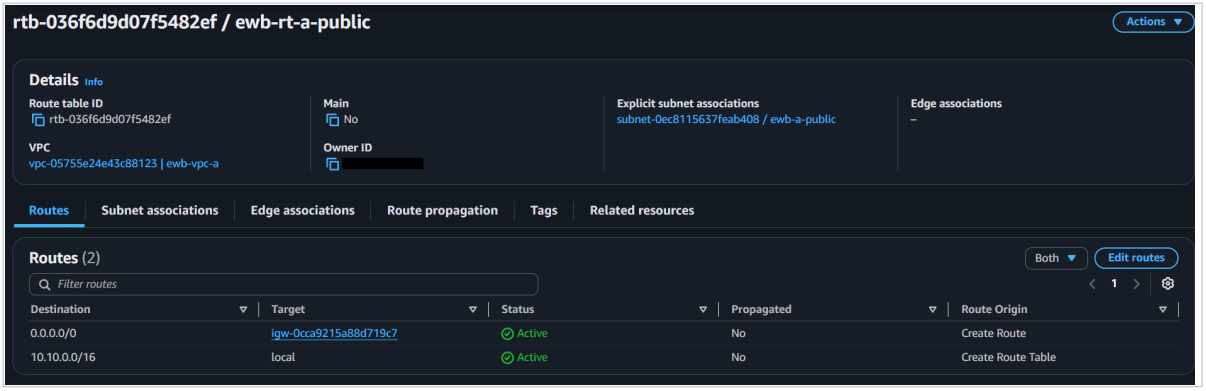


Figure 44 (C14d3) | **Host / console:** ewb-rt-a-public, Routes | **Test:** peering route removed | **Result:** 0.0.0.0/0 to igw and 10.10.0.0/16 local remain

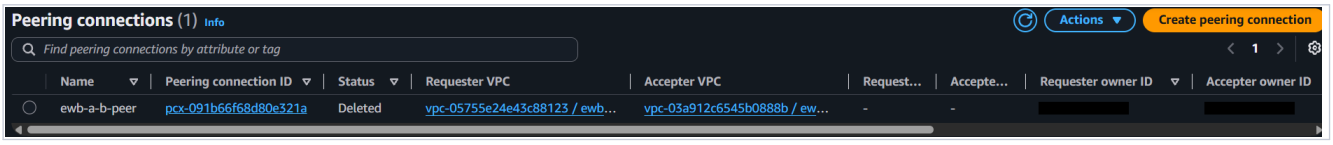


Figure 45 (C14e) | **Host / console:** VPC, Peering connections | **Test:** peering connection deleted | **Result:** ewb-a-b-peer shows Deleted

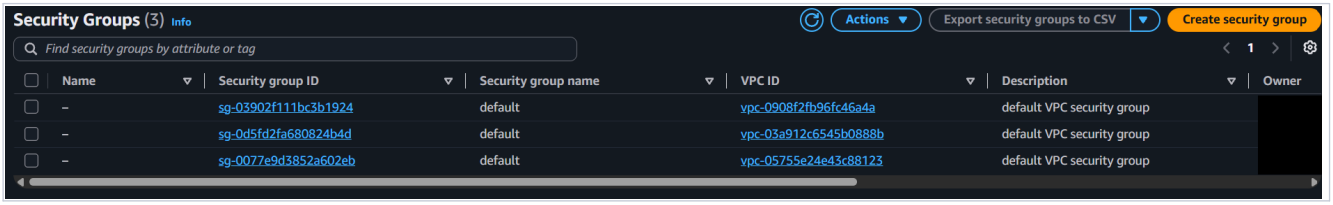


Figure 46 (C14sg) | **Host / console:** VPC, Security groups | **Test:** lab security groups deleted | **Result:** only the three default groups (one per VPC) remain; the owner column is blacked out

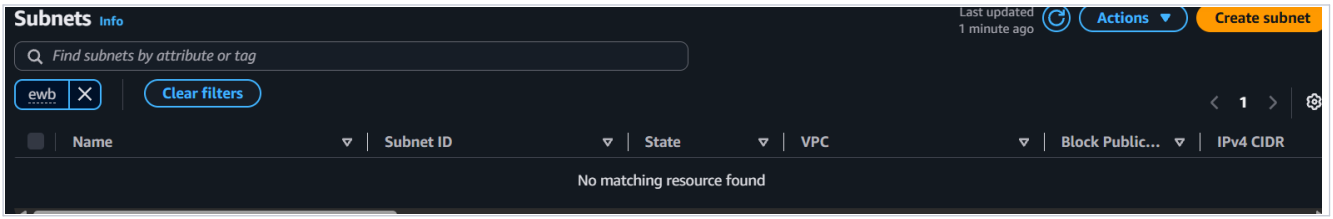


Figure 47 (C14subnets) | **Host / console:** VPC, Subnets (filter: ewb) | **Test:** subnets deleted | **Result:** No matching resource found

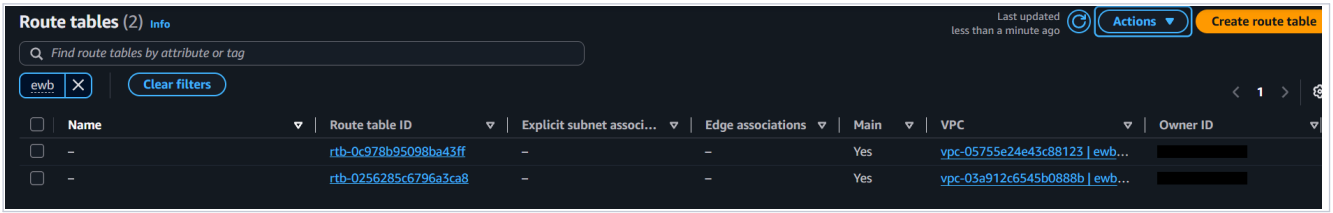


Figure 48 (C14rt) | **Host / console:** VPC, Route tables (filter: ewb) | **Test:** custom route tables deleted | **Result:** only the two unnamed main route tables of the lab VPCs remain; they are removed together with the VPCs

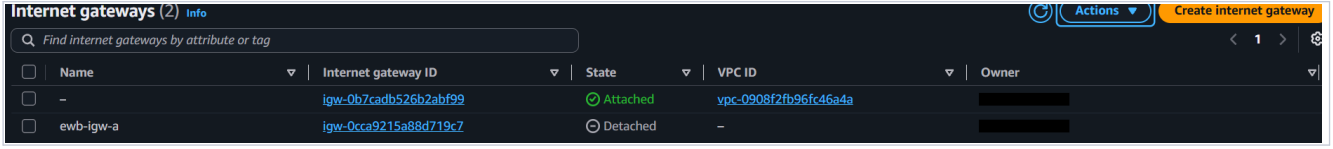


Figure 49 (C14fdet) | **Host / console:** VPC, Internet gateways | **Test:** ewb-igw-a detached | **Result:** State Detached

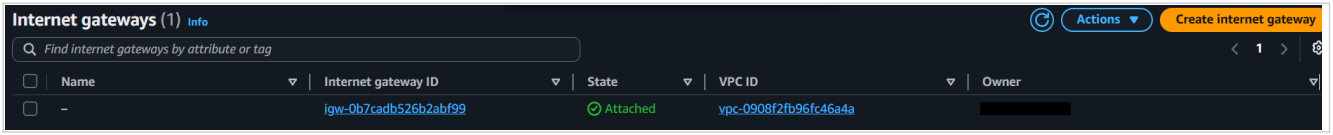


Figure 50 (C14f) | **Host / console:** VPC, Internet gateways | **Test:** ewb-igw-a deleted | **Result:** only the default VPC's gateway remains

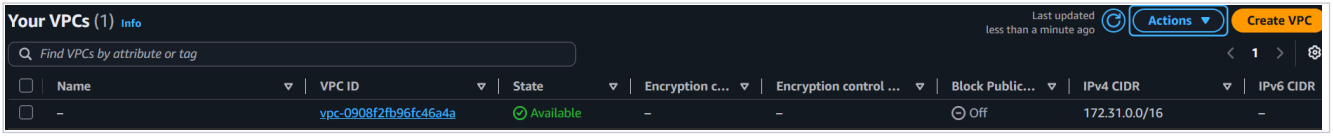


Figure 51 (C14g) | **Host / console:** VPC, Your VPCs | **Test:** both lab VPCs deleted | **Result:** only the default VPC 172.31.0.0/16 remains

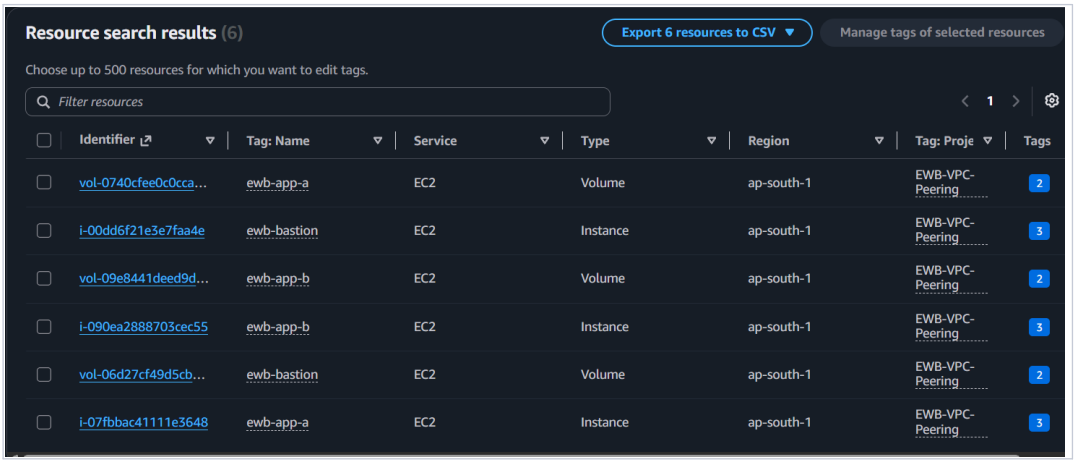


Figure 52 (C14h) | **Host / console:** Resource Groups and Tag Editor (Project = EWB-VPC-Peering, ap-south-1) | **Test:** tag check | **Result:** the search still lists the three terminated instances and their volumes; AWS keeps deleted resources in this inventory for a while, and EC2 shows the instances Terminated (C14a) and no volumes (C14b)

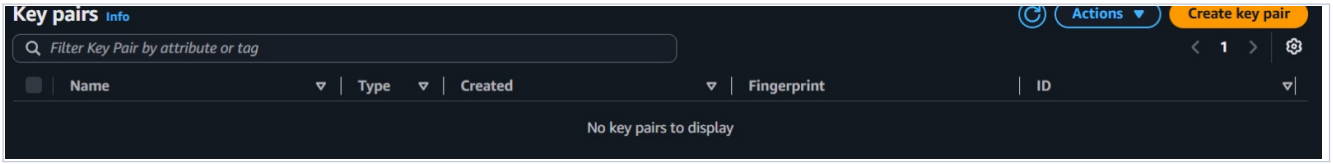


Figure 53 (C14i) | **Host / console:** EC2, Key pairs | **Test:** key pair deleted | **Result:** No key pairs to display

13. What I learned

• VPC peering is only the link. To make traffic flow I needed routes on both sides, the correct security group rules, and (by default) allow-all network ACLs.
• CIDR planning comes first: two VPC ranges that overlap cannot be peered, and splitting them into different subnets does not help.
• A public subnet needs both an internet-gateway route and an instance with a public IP. Private instances can be reached safely through a bastion with SSH ProxyJump, without copying the private key anywhere.
• Security groups are stateful, so I only had to allow the request direction. Using /32 sources limited each rule to exactly one lab host.
• The controlled failure showed me how to separate a routing problem from a security group problem: the peering and routes were healthy, and removing one rule alone stopped the ping.
• Troubleshooting notes: a path with spaces must be quoted in WSL, a "host identification has changed" warning came from an old saved key for the same private IP, and AWS refuses to delete a route table or VPC while subnets or other resources still depend on it, so cleanup has to follow dependency order.
• Tagging every resource with Project and Student made it easy to check that nothing was left running, and terminating the instances (not just stopping them) removed the disks and the cost.

Appendix A. Configuration screens

Creation forms captured while building (shown before clicking Create or Launch). Account ID and home IP are blacked out.

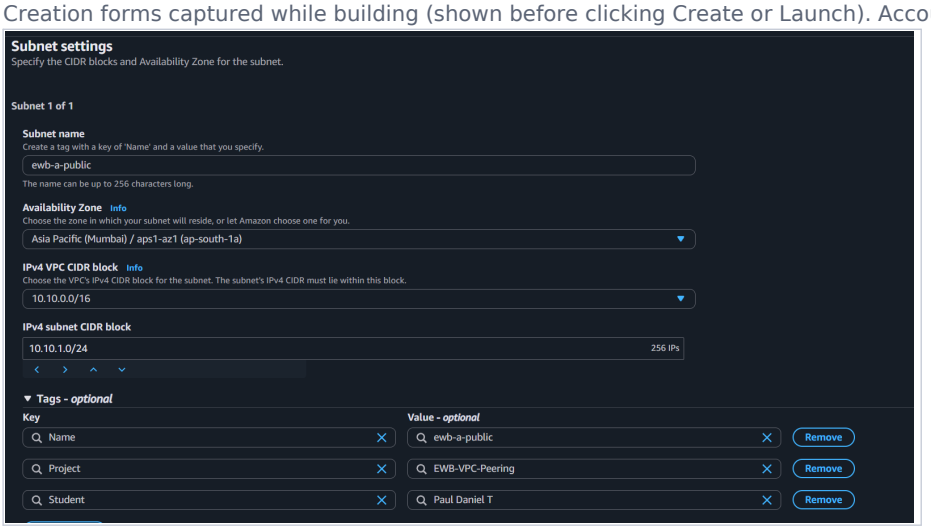


Figure 54 (S2a) | **Host / console:** Create subnet | **Test:** ewb-a-public | **Result:** 10.10.1.0/24 in ap-south-1a, VPC A 10.10.0.0/16, tags Name / Project / Student

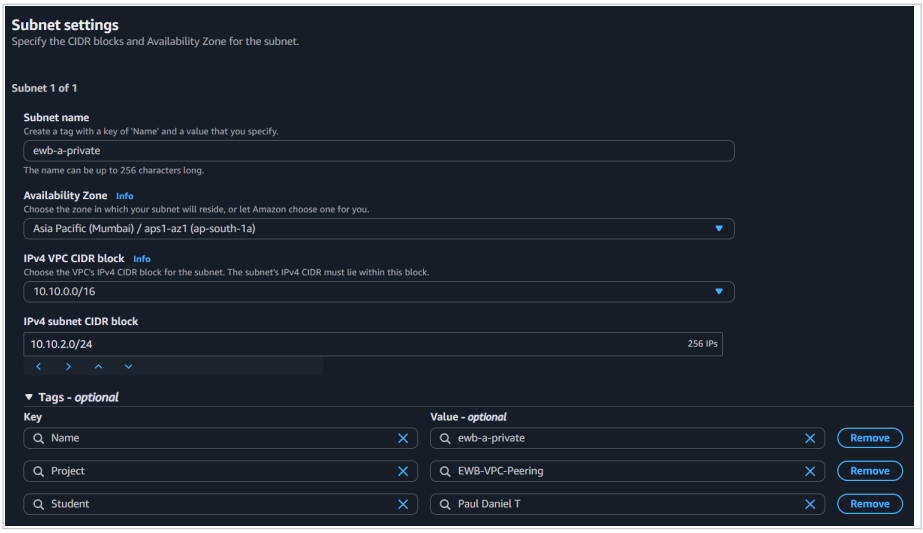


Figure 55 (S2b) | **Host / console:** Create subnet | **Test:** ewb-a-private | **Result:** 10.10.2.0/24 in ap-south-1a, VPC A, tags set

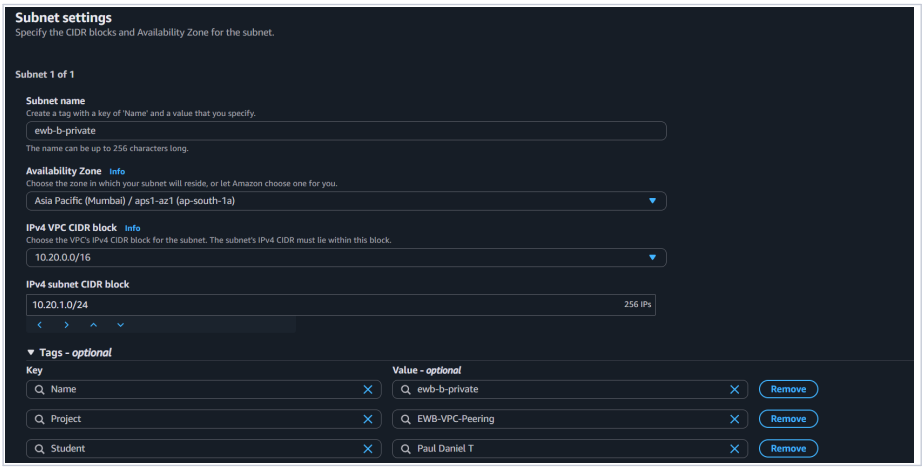


Figure 56 (S2c) | **Host / console:** Create subnet | **Test:** ewb-b-private | **Result:** 10.20.1.0/24 in ap-south-1a, VPC B 10.20.0.0/16, tags set

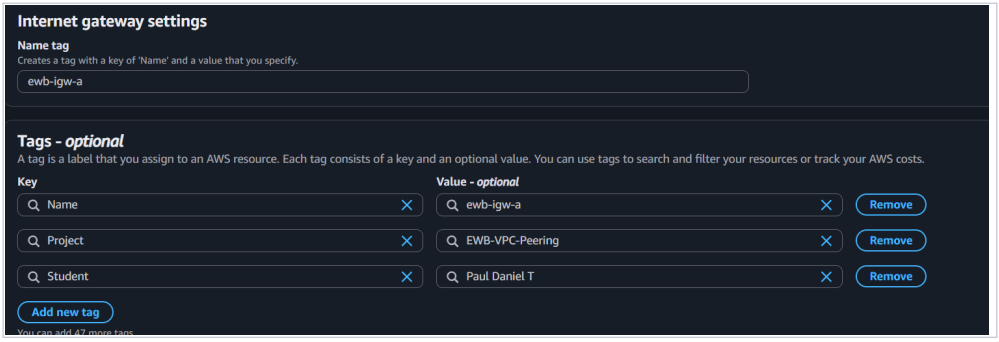


Figure 57 (S3a) | **Host / console:** Create internet gateway | **Test:** ewb-igw-a | **Result:** name and Project / Student tags set

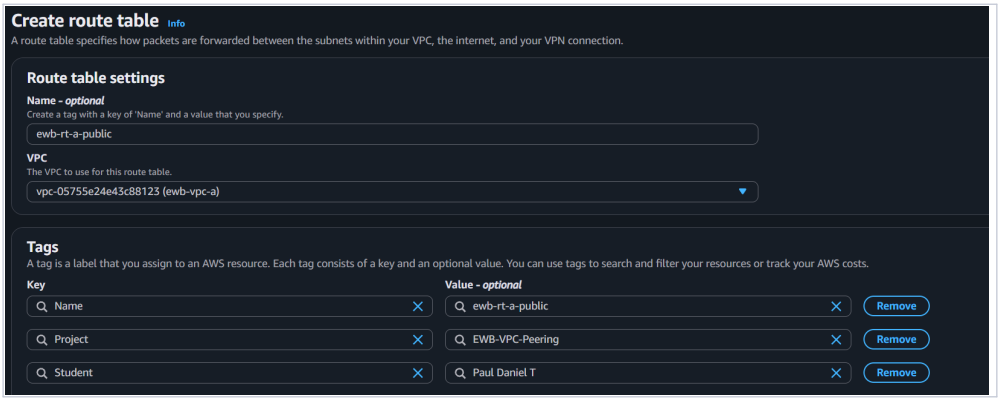


Figure 58 (S4form) | **Host / console:** Create route table | **Test:** ewb-rt-a-public | **Result:** VPC ewb-vpc-a (vpc-05755...), tags set

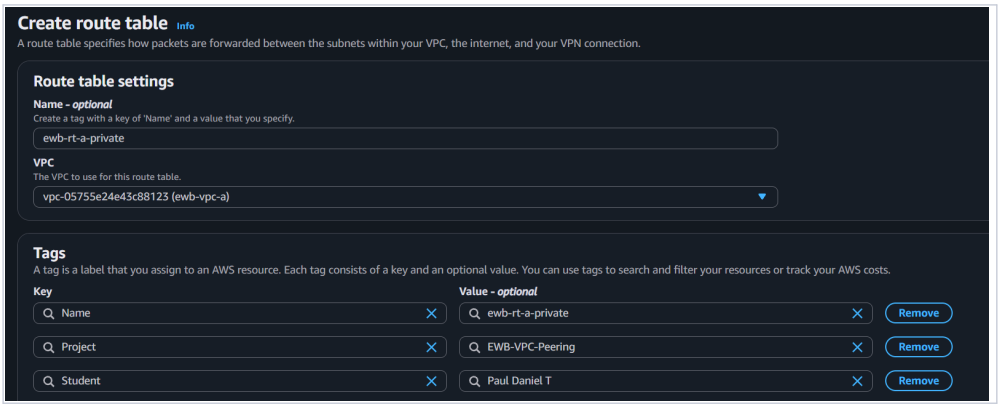


Figure 59 (S5forma) | **Host / console:** Create route table | **Test:** ewb-rt-a-private | **Result:** VPC ewb-vpc-a (vpc-05755...), tags set

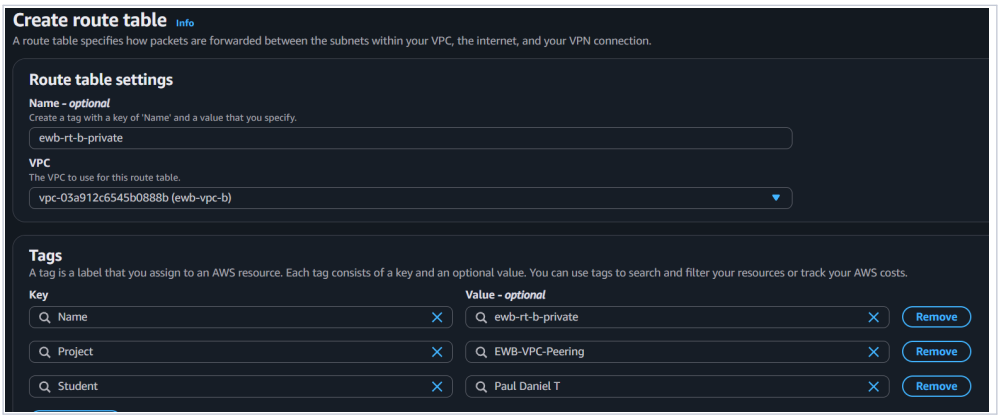


Figure 60 (S5formb) | **Host / console:** Create route table | **Test:** ewb-rt-b-private | **Result:** VPC ewb-vpc-b (vpc-03a91...), tags set

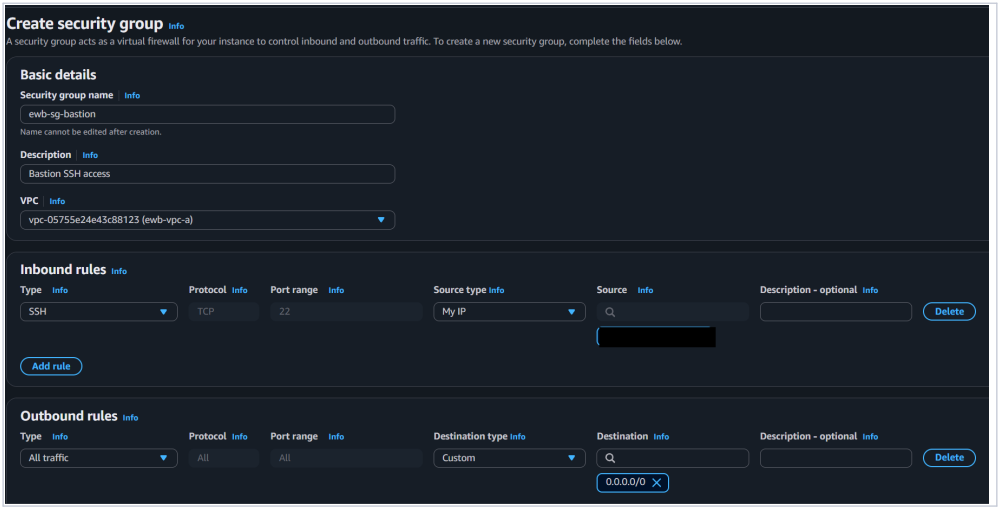


Figure 61 (S6aform) | **Host / console:** Create security group | **Test:** ewb-sg-bastion | **Result:** VPC A; inbound SSH 22 from My IP /32 (blacked out); outbound default

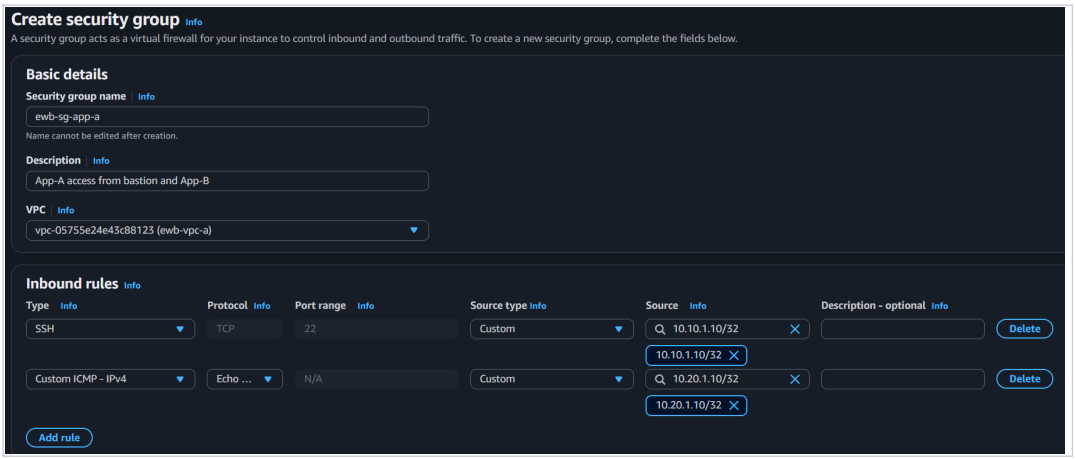


Figure 62 (S6bform) | **Host / console:** Create security group | **Test:** ewb-sg-app-a | **Result:** VPC A; SSH from 10.10.1.10/32 and ICMP Echo Request from 10.20.1.10/32

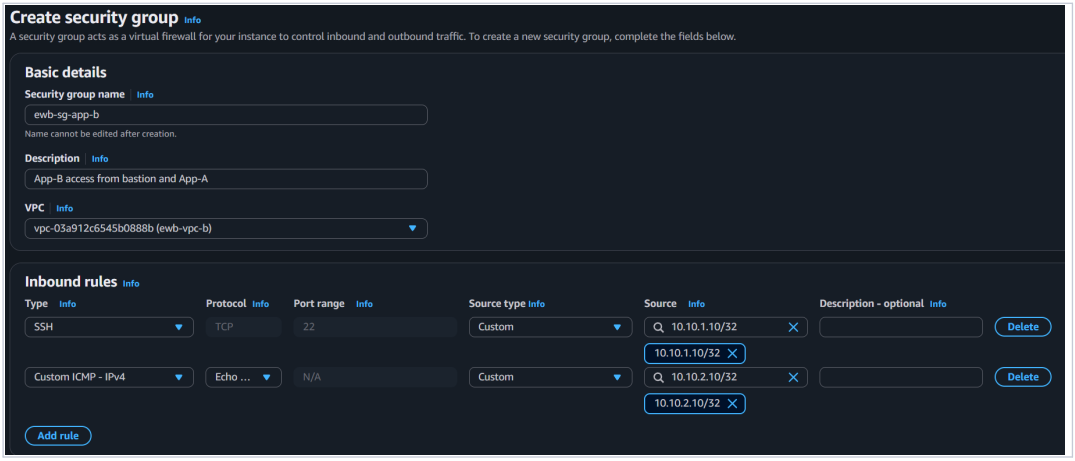


Figure 63 (S6cform) | **Host / console:** Create security group | **Test:** ewb-sg-app-b | **Result:** VPC B (vpc-03a91...); SSH from 10.10.1.10/32 and ICMP Echo Request from 10.10.2.10/32

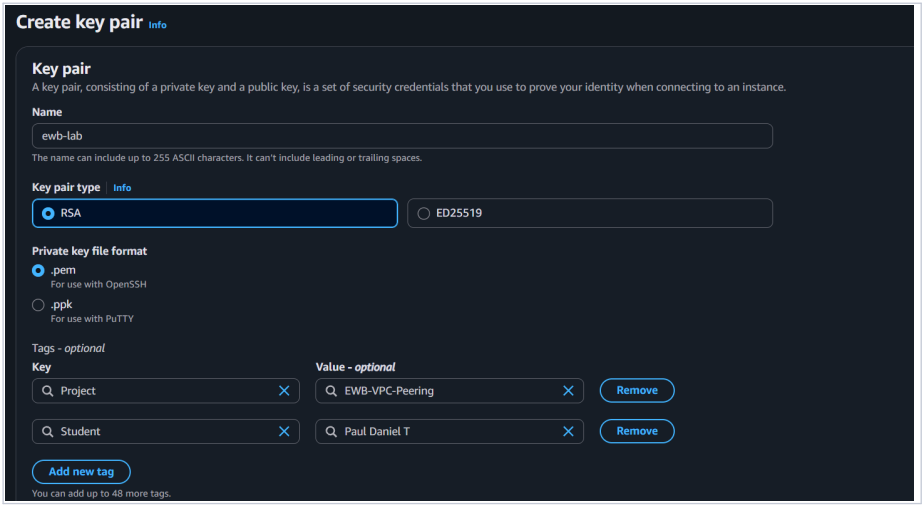


Figure 64 (S7aform) | **Host / console:** Create key pair | **Test:** ewb-lab | **Result:** RSA, .pem for OpenSSH, tags set

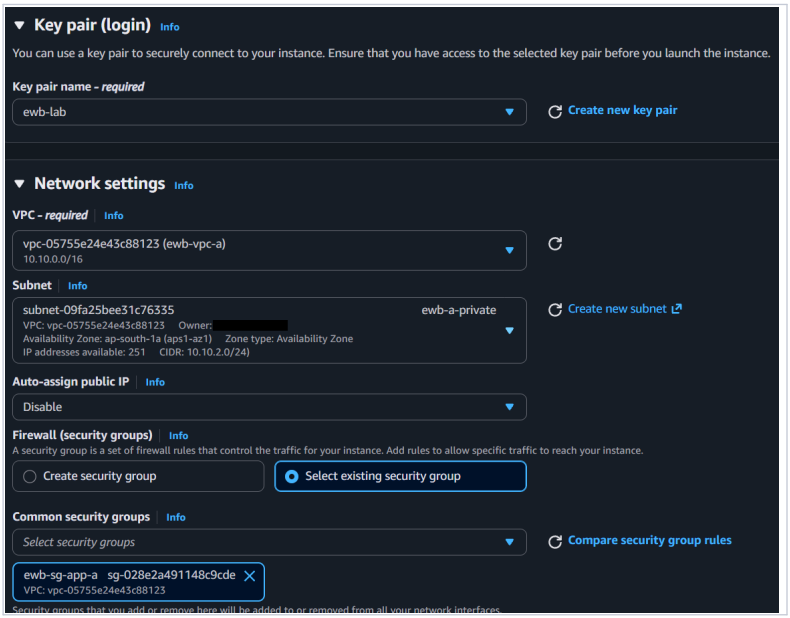


Figure 65 (S8a) | **Host / console:** Launch instance (App-A) | **Test:** network settings | **Result:** VPC ewb-vpc-a, subnet ewb-a-private, auto-assign public IP Disable, security group ewb-sg-app-a

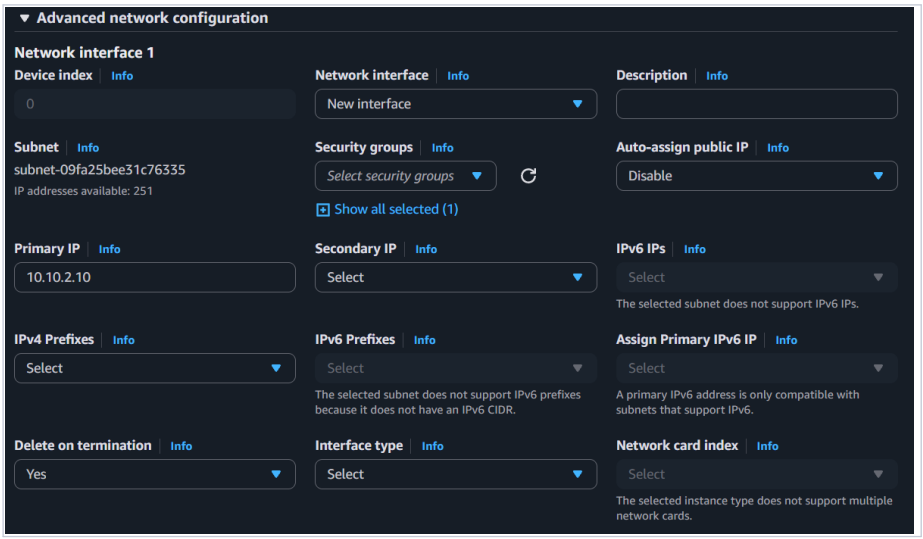


Figure 66 (S8a2) | **Host / console:** Launch instance (App-A) | **Test:** advanced network configuration | **Result:** primary IP 10.10.2.10, auto-assign public IP Disable, delete on termination Yes

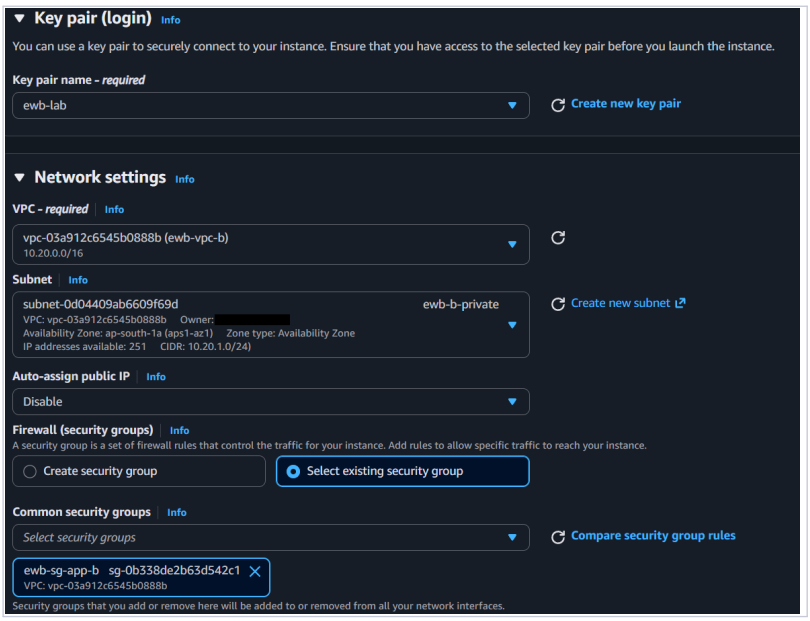


Figure 67 (S8b) | **Host / console:** Launch instance (App-B) | **Test:** network settings | **Result:** VPC ewb-vpc-b, subnet ewb-b-private, auto-assign public IP Disable, security group ewb-sg-app-b

Appendix B. Instance details

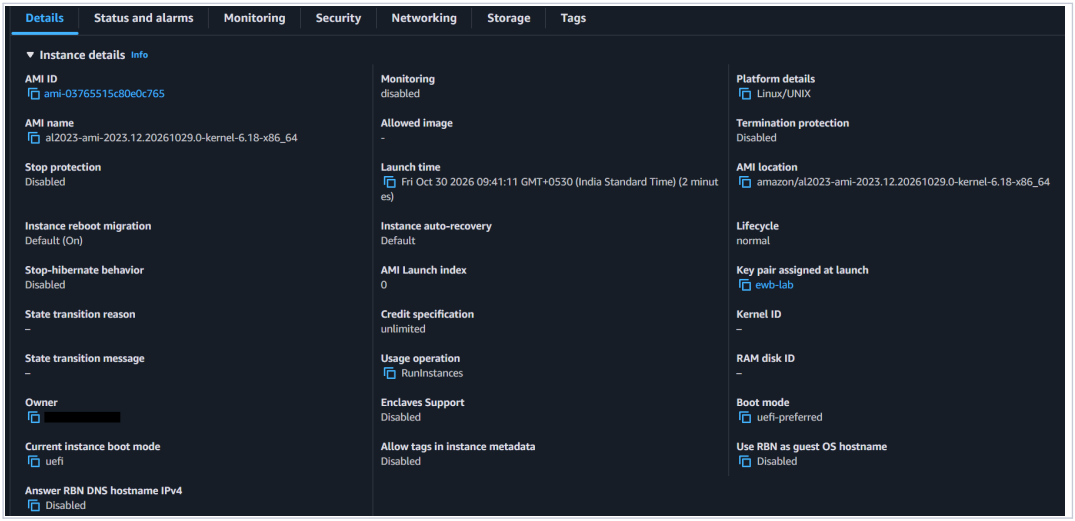


Figure 68 (S7d) | **Host / console:** ewb-bastion, Details | **Test:** AMI and key pair | **Result:** Amazon Linux 2023 AMI, key pair ewb-lab, launched 30 Oct 2026 09:41 IST

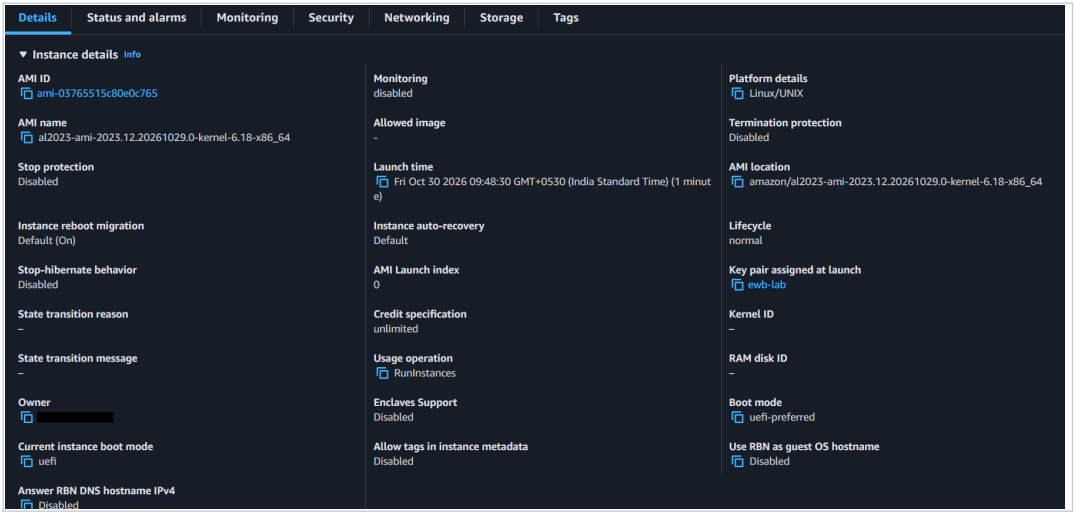


Figure 69 (S8a4) | **Host / console:** ewb-app-a, Details | **Test:** AMI and key pair | **Result:** same AMI and key pair, launched 30 Oct 2026 09:48 IST

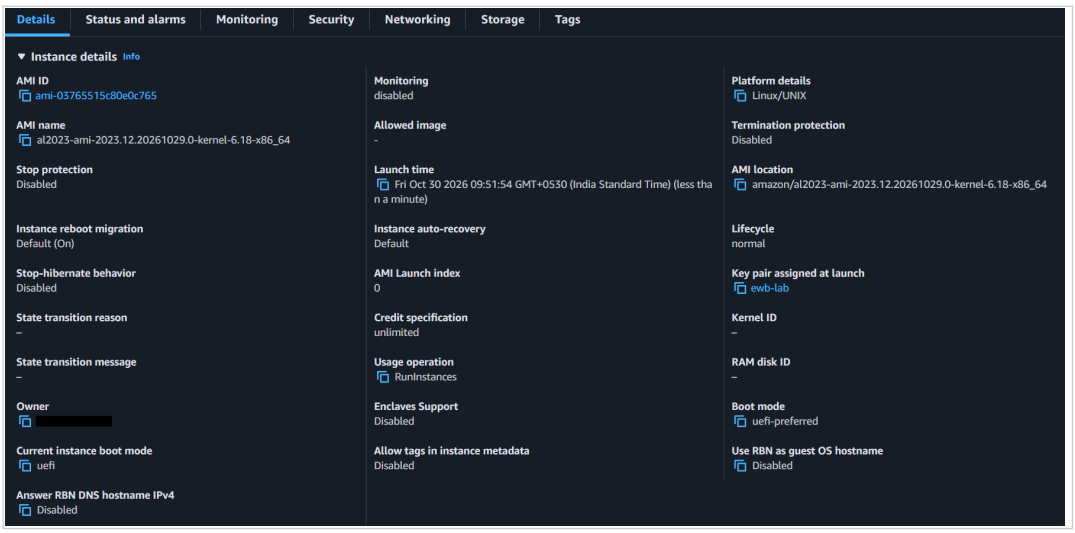


Figure 70 (S8b4) | **Host / console:** ewb-app-b, Details | **Test:** AMI and key pair | **Result:** same AMI and key pair, launched 30 Oct 2026 09:51 IST